PART IV

Microservices Architecture Foundations

Definitions, Characteristics, Benefits, Challenges, and a Gallery of Reference Architectures — Before You Design Your Own

Java / Spring Boot | C# / ASP.NET | Node.js | Python / Django / Flask

MongoDB | Redis | Cassandra | PostgreSQL / MySQL / SQL Server

Table of Contents

1. What Are Microservices? 3
2. Benefits of Microservices Architecture 5
3. Challenges of Microservices Architecture 7
4. A Gallery of Reference Architectures 9
5. Case Study: A Java Spring Boot Reference Implementation 12
6. Designing Our E-Commerce Microservices Architecture 15
7. Technology Stack Choices 17
8. Resilience Patterns: Circuit Breaker, Retry, Rate Limiter & Bulkhead 18
9. Rate Limiter Algorithms — A Practical Approach 21
10. Caching Strategies: Caffeine & Redis with Eviction Policies 25
11. Docker & Kubernetes: Architecture, Components & Core Operations 29
12. CI/CD Pipeline: Canary & Blue-Green Deployment Strategies 34
13. What's Next 37

1. What Are Microservices?

A Working Definition

Microservices are small, independently deployable business services that work together over the network rather than in a single shared process. Martin Fowler's widely cited description frames it well: the microservice style is an approach to building a single application as a suite of small services, each running in its own process and communicating through lightweight mechanisms — most often an HTTP or gRPC API. Each of those services is built around a specific business capability, deployable through a fully automated pipeline, with minimal centralized coordination — which is exactly what allows different services to be written in different languages and backed by different data stores without friction.

Put another way: microservices architecture is a cloud-native approach where an application is composed of many small, loosely coupled, independently deployable components, each of which:

- owns its own technology stack, including its database and data management model;
- communicates with other services over a combination of REST APIs, event streaming, and message brokers (the full spectrum covered hands-on in Part III);
- is organized around a business capability, with the boundary between services usually referred to as a **bounded context**.

Core Characteristics

Characteristic	What it means in practice
Small, independent, loosely coupled	A single small team can write, own, and maintain one service as its own separate codebase
Independently deployable	A team ships an update to their service without rebuilding or redeploying the rest of the application
Owns its own persistence	Each service is responsible for its own data — the Database per Microservice pattern from Part I, not a shared data layer
Communicates via well-defined APIs	Internal implementation details stay hidden behind the API; callers depend only on the contract, not the internals
No forced shared stack	Services don't need to share the same language, libraries, or frameworks as their neighbors

Fowler's Broader Organizational Characteristics

Beyond the technical shape, Fowler's original article also names several characteristics that are as much about how organizations work as how software is structured:

- **Componentization via Services** — a component becomes a unit of software that can be independently replaced and upgraded, rather than a library statically linked into one big deployable.
- **Organized around Business Capabilities** — services are split by what the business does (Catalog, Ordering, Discount) rather than by technical layer (UI, business logic, data access).
- **Products, not Projects** — the "you build it, you run it" model popularized by Amazon, where a team owns its service's full lifecycle in production rather than handing it off after launch.
- **Smart Endpoints, Dumb Pipes** — services own their own domain logic and expose it over simple, RESTful interfaces, rather than routing complex business rules through a smart middleware/ESB layer.
- **Decentralized Governance** — teams are free to choose their own tools and share proven approaches as reusable libraries (Fowler cites Netflix as the canonical example) rather than a single centralized architecture board mandating one stack for everyone.
- **Decentralized Data Management** — often called **polyglot persistence**: each service picks the database technology that fits its own data, rather than every service sharing one central database.
- **Infrastructure Automation** — deployment to every environment is automated per service, not a manual, centrally coordinated release process.
- **Design for Failure** — services are built assuming their dependencies will sometimes fail, and handle that gracefully (this is exactly what Part I's Circuit Breaker pattern formalizes).

> **Why this framing matters before writing any code:** every pattern across Parts I–III of this guide is, in one way or another, an implementation of one of these characteristics — Database per Microservice implements Decentralized Data Management, Circuit Breaker implements Design for Failure, API Gateway and BFF implement Smart Endpoints/Dumb Pipes at the edge, and so on. Understanding the characteristics first makes it much clearer *why* each pattern exists, rather than just *how* to implement it.

Hands-On Deep Dive: A Minimal Microservice Embodying These Characteristics

Rather than describe these characteristics only in the abstract, here's the smallest possible Spring Boot service that actually embodies them — its own codebase, its own database, its own API, deployable and scalable independently of anything else.

`pom.xml — self-contained, no shared library dependency on any other service`

```
<dependencies>
  <dependency>
```

```
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-mongodb</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-actuator</artifactId>   <!-- health, for Infrastructure Automation -->
  </dependency>
</dependencies>
```

CatalogServiceApplication.java — "Smart Endpoint": all domain logic lives here, nowhere else

```
@SpringBootApplication
public class CatalogServiceApplication {
    public static void main(String[] args) { SpringApplication.run(CatalogServiceApplication.class, args); }
}

@RestController
@RequestMapping("/v1/products")
class ProductController {
    private final ProductRepository repository;   // this service's OWN database -- no other service touches it

    @GetMapping("/{id}")
    public Product get(@PathVariable String id) {
        return repository.findById(id).orElseThrow(() -> new ProductNotFoundException(id));
    }
    // "dumb pipe": a plain RESTful endpoint -- no smart middleware/ESB routing business logic through it
}
```

Dockerfile — Infrastructure Automation: this service builds and deploys on its own

```
FROM eclipse-temurin:17-jre
COPY target/catalog-service.jar app.jar
EXPOSE 8081
ENTRYPOINT ["java", "-jar", "/app.jar"]
```

> **What this tiny example already proves:** this service can be built, tested, containerized, scaled, and redeployed without touching any other service's code, pipeline, or database — the smallest possible demonstration of independent deployability, decentralized data management, and smart endpoints/dumb pipes, all at once. Every larger pattern in Parts I–III builds on exactly this foundation.

2. Benefits of Microservices Architecture

Agility

Because services are smaller and independently deployable, bug fixes and feature releases are far easier to manage. A single service can be updated — and rolled back if something goes wrong — without touching the rest of the application. In a monolith, a bug found anywhere can block the entire release pipeline, forcing unrelated features to wait on that one fix.

Small, Focused Teams

A microservice should be small enough for one feature team to build, test, and deploy on its own. This lets an organization form small, cross-functional teams around a service or a small group of related services, operating with the agility that comes from small team size — larger teams tend to slow down as communication overhead and coordination cost rise.

Small, Separated Codebases

In a monolith, the codebase inevitably grows large enough that dependencies between unrelated parts become tangled, and adding one new feature can require refactoring far beyond where that feature actually lives. Since microservices don't share code or a data store with their neighbors, dependencies stay minimal by construction, which keeps adding new features comparatively cheap even as the overall system grows.

The Right Tool for Each Job

A traditional layered application typically shares one stack and one large relational database across the entire application — even where a different tool would clearly serve a particular module better. Microservices remove that constraint: each team can pick the stack that best fits its specific service, and because services communicate over REST, event streaming, or message brokers rather than shared code, the stack of any individual service can evolve independently as better tools become available, without forcing a platform-wide migration.

Fault Isolation

Loose coupling between services also brings fault isolation: if one microservice becomes unavailable, it doesn't have to take the rest of the application down with it — provided the system is actually designed for that outcome, using patterns like Retry and Circuit Breaking (Part I, Pattern 8) to contain failures rather than let them cascade. Handled well, a failure in one service resolves without customers ever noticing a business impact.

Scalability

Because each service scales independently, you can scale out exactly the sub-services under heavy load without scaling the entire application — meaningfully reducing wasted infrastructure compared to scaling a monolith end-to-end for the sake of one hot module. Paired with a container orchestrator like Kubernetes, this also means packing a higher density of services onto the same hardware, improving overall resource utilization.

Data Isolation

Following the Database per Microservice pattern (Part I, Pattern 1) means schema changes only ever affect the one database behind the service being changed. In a monolith sharing a single schema, even a small schema update can become a risky, coordinated, all-hands change across every module that touches that table.

> **The common thread across all of these benefits:** independence. Independent deployability drives agility; independent team ownership drives focus; independent stacks drive the right-tool-for-the-job flexibility; independent failure domains drive fault isolation; independent scaling drives efficient resource use; independent data ownership drives safe schema evolution. Every benefit here traces back to the same root cause — decoupling services from each other as completely as the business domain allows.

Hands-On Deep Dive: Independent Scaling and Independent Deployment in Practice

"Scale only the service that needs it" and "deploy one service without touching the rest" are easy to state and easy to actually demonstrate — this is where the benefit becomes visible as running infrastructure rather than a talking point.

`docker-compose.yml — scaling only Catalog, since it's the one under heavy read load`

```
docker compose up -d --scale catalog-service=5 --scale ordering-service=1
# Catalog gets 5 instances behind the gateway's load balancer; Ordering stays at 1 --
# no change to Ordering's deployment, code, or database at all
```

`catalog-deployment.yaml — the same idea, expressed as a Kubernetes HPA (Part II, Section 10)`

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: catalog-service-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: catalog-service
  minReplicas: 2
  maxReplicas: 15   # Catalog can burst to 15 pods under load
  metrics:
    - type: Resource
      resource: { name: cpu, target: { type: Utilization, averageUtilization: 65 } }
---
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: ordering-service-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: ordering-service
  minReplicas: 2
  maxReplicas: 4    # Ordering's write volume rarely justifies more than a handful of pods
```

Deploying one service's update, live, with zero coordination with any other service

```
# build and push only the changed service
mvn -pl discount-service clean package
docker build -t registry.example.com/discount-service:1.4.0 discount-service/
docker push registry.example.com/discount-service:1.4.0

# rolling update -- Kubernetes replaces pods gradually, other services are entirely untouched
kubectl set image deployment/discount-service discount-service=registry.example.com/discount-service:1.4.0
kubectl rollout status deployment/discount-service

# instant rollback if something's wrong -- again, without touching any other service
kubectl rollout undo deployment/discount-service
```

> **What to notice here:** at no point does scaling or deploying Discount or Catalog require a build, a test run, or a deploy of Ordering, Basket, or the Gateway. That absence of coordination — not any single command above — is the actual benefit; the commands are just what that benefit looks like once it's running.

3. Challenges of Microservices Architecture

Microservices bring real, significant benefits — but moving from a monolith to microservices also brings real, significant management complexity. These challenges don't stop organizations from adopting microservices, but they do need to be planned for deliberately rather than discovered the hard way.

Complexity

A microservices application has many services that need to work together to create business value — more moving parts than an equivalent monolith, even though each individual service is simpler on its own. Deployments across many services, each potentially on its own release cadence, become harder to track and version. Communication itself becomes a genuinely hard topic: in a monolith, calling another module is an in-process method call on the same machine; in microservices, it's inter-service communication over a network, potentially across different geographic regions, and needs an explicit strategy (exactly what Part III of this guide is dedicated to).

Network Problems and Latency

Because microservices communicate over the network, every inter-service call is subject to network problems that simply don't exist for an in-process call. A chain of several service calls for one client request compounds latency and risk with every additional hop (Part I's Bonus A section covers this "Chain Queries" problem and its Service Aggregator fix in depth) — proper API design and, where appropriate, asynchronous communication via message brokers (Part I Bonus B, Part III Section 8) are both necessary tools for keeping this under control.

Development and Testing

Consider a single business requirement that spans several microservices but needs to behave as one coherent flow from the user's perspective. Testing that end-to-end journey is meaningfully harder in microservices than in a monolith, where the same flow runs in one process and can be exercised with a single integration test. Tooling doesn't always model service-to-service dependencies well, and refactoring across a service boundary — as opposed to refactoring within one module of a monolith — is inherently more disruptive to coordinate.

Data Integrity

Since each microservice owns its own persistence, maintaining data consistency across services is a genuine challenge rather than a given. Most systems lean on eventual consistency wherever the business can tolerate it (Part I Pattern 2, Part II Sections 4–5), but any operation that would have been a single ACID transaction in a monolith needs a deliberate strategy in microservices — this is exactly the gap Part I's Saga pattern (Pattern 4) exists to fill.

Challenge	Where this guide addresses it
Complex inter-service communication	Part III — Sync vs. Async, REST vs. gRPC
Chatty, chained calls increasing latency	Part I, Bonus A — Service Aggregator & Service Registry
Cascading failures from network problems	Part I, Pattern 8 — Circuit Breaker
Cross-service transactions / data integrity	Part I, Pattern 4 — Saga Pattern
Reliable event publishing despite partial failures	Part I, Bonus D — The Outbox Pattern
Read/write workload mismatch across services	Part I, Pattern 2 — CQRS

> **The honest framing this guide uses throughout:** none of these challenges are reasons to avoid microservices outright — they're reasons to adopt microservices deliberately, pattern by pattern, only once a monolith's limits are actually being felt (see Part I Section 1's Architecture Evolution Journey and Capacity Planning tables). Every challenge listed above has a corresponding, well-established pattern; the skill is knowing which pattern to reach for, and when.

Hands-On Deep Dive: Making Cross-Service Complexity Observable

The Complexity and Network Problems challenges above share a root cause: once a request crosses several services, you lose the simple "read the stack trace" debugging model a monolith gives you for free. Distributed tracing is the concrete tool that gets that visibility back — every hop of a request gets tagged with the same correlation id, so the full journey can be reconstructed after the fact.

pom.xml — Micrometer Tracing + Zipkin exporter, added to every service

```
<dependency>
  <groupId>io.micrometer</groupId>
  <artifactId>micrometer-tracing-bridge-brave</artifactId>
</dependency>
<dependency>
  <groupId>io.zipkin.reporter2</groupId>
```

```
  <artifactId>zipkin-reporter-brave</artifactId>
</dependency>
```

application.yml — identical config dropped into every service in the chain

```
management:
  tracing:
    sampling:
      probability: 1.0   # trace every request in dev; sample a fraction in production
  zipkin:
    tracing:
      endpoint: http://zipkin:9411/api/v2/spans
```

With this in place, every log line across every service automatically carries the same `traceId`, and the correlation id propagates transparently through both HTTP headers and Kafka message headers — no manual plumbing required in the request-handling code itself.

Seeing the payoff: one Kafka consumer, logging with trace context for free

```
@KafkaListener(topics = "order-events")
public void onOrderEvent(OrderCreatedEvent event) {
    log.info("Projecting order {} into read model", event.orderId());
    // this log line automatically includes the SAME traceId as the original
    // "POST /orders" HTTP request that triggered this event, several hops upstream
}
```

```
# querying Zipkin to see the full cross-service journey for one request
curl "http://localhost:9411/api/v2/trace/{traceId}" | jq '.[] | {service: .localEndpoint.serviceName, name, duration}'
# returns every span -- Gateway -> Ordering -> Kafka publish -> Ordering-read-model-projector --
# in one ordered list, with per-hop timing, instead of grepping five separate log files by hand
```

> **The point of this exercise:** distributed tracing doesn't remove the Complexity or Network Problems challenges — it makes them *debuggable*. Once a platform has more than a handful of services, skipping this setup isn't a shortcut; it just moves the cost from "five minutes adding a dependency to every service" to "hours grepping through unrelated log files during an incident."

4. A Gallery of Reference Architectures

Before designing a single architecture from scratch, it's worth looking at how other real systems have already solved this problem — the same handful of building blocks (API Gateway, message broker, service discovery, identity provider) show up again and again, just combined and matured differently depending on scale and requirements. The four reference architectures below progress roughly from simplest to most mature.

Reference Architecture 1 — The Simplest Shape

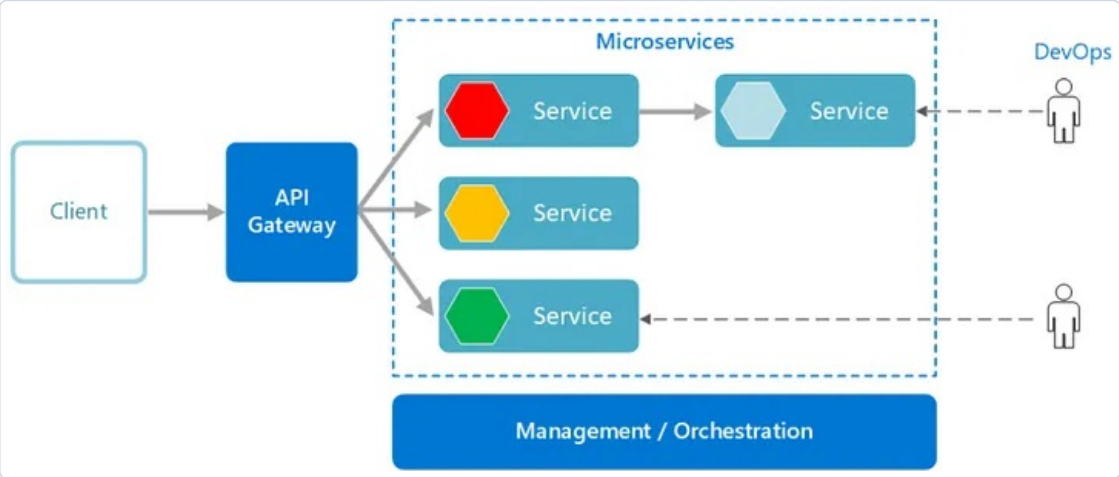


Fig. 1 — A Client talks to one API Gateway, which fans out to several Services. One of those services calls another directly, and a DevOps role manages/deploys services independently — all sitting under one Management/Orchestration layer.

This is about as minimal as a microservices reference architecture gets: one API Gateway in front of several independently deployable services, with a management/orchestration layer underneath handling deployment. There's no message broker yet — any service-to-service communication here is synchronous (Part III, Sections 3–5).

Reference Architecture 2 — Adding a Message Broker

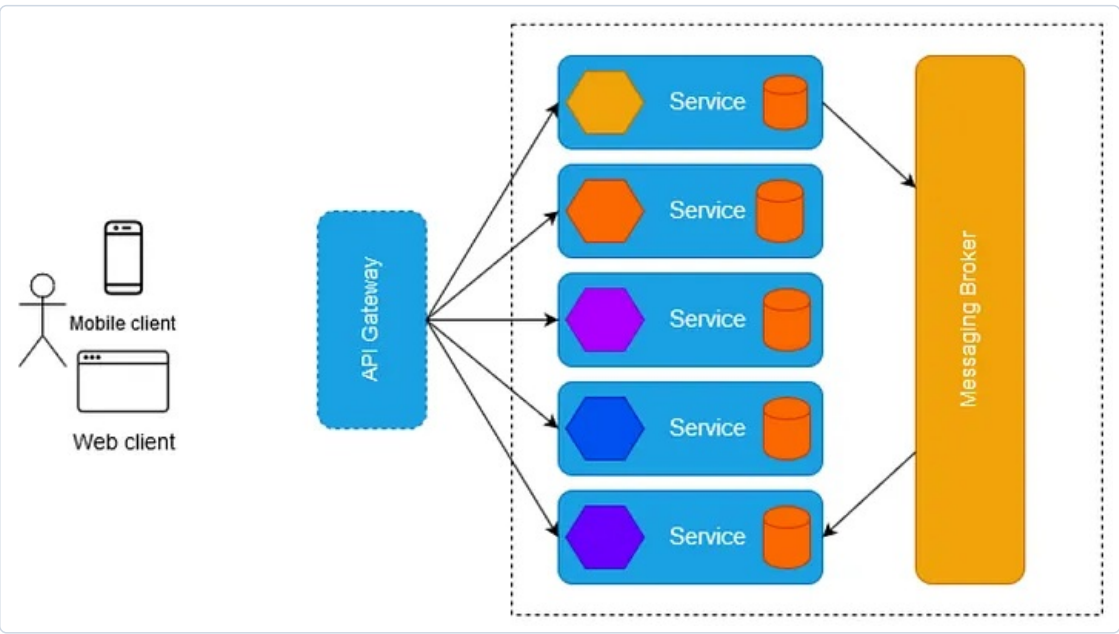


Fig. 2 — Mobile and Web clients call a single API Gateway, which routes to five services, each with its own database. A Messaging Broker now sits alongside the services, letting them communicate asynchronously with each other in addition to the gateway's synchronous routing.

This is the most common shape referenced across the industry: an API Gateway for client-facing synchronous traffic, each service still owning its own database (Part I, Pattern 1), and a message broker enabling asynchronous, decoupled communication between the services themselves (Part I, Bonus B; Part III, Section 8) — rather than forcing every inter-service interaction through direct, synchronous calls.

Reference Architecture 3 — Applied to E-Commerce

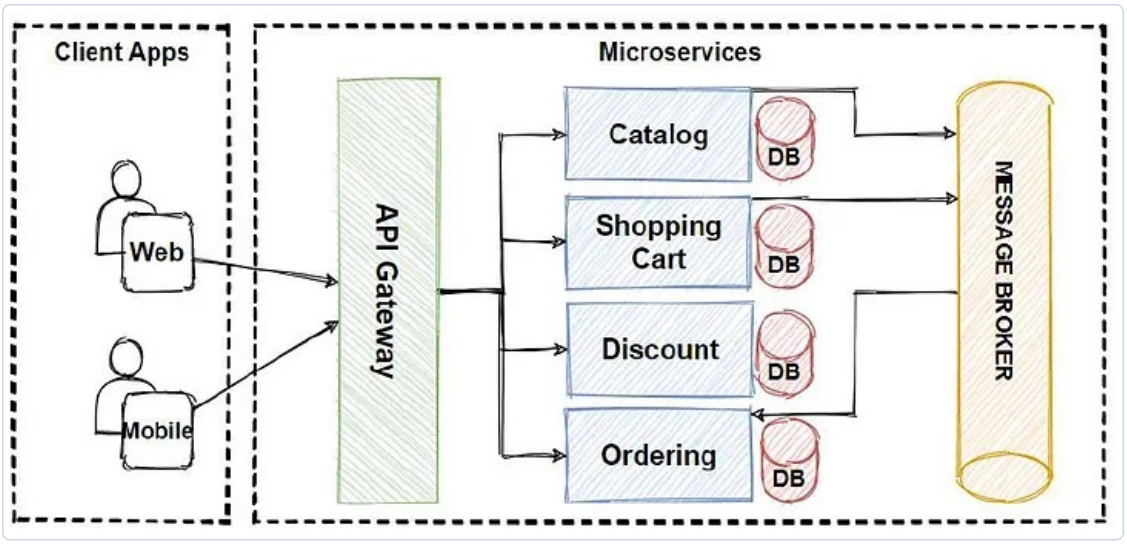


Fig. 3 — The same shape as Fig. 2, now with concrete e-commerce services: Catalog, Shopping Cart, Discount, and Ordering, each with its own DB, all reachable through one API Gateway and all wired into a shared Message Broker.

This is the exact reference shape this entire guide series evolves from — it's the starting architecture in Part I before layering on CQRS, Event Sourcing, Sagas, and the rest of the pattern catalog. Recognizing this shape is often the fastest way to orient yourself in someone else's unfamiliar microservices codebase: find the API Gateway, find the message broker, and the rest of the topology usually falls into place.

Reference Architecture 4 — A More Mature, Production-Hardened Shape

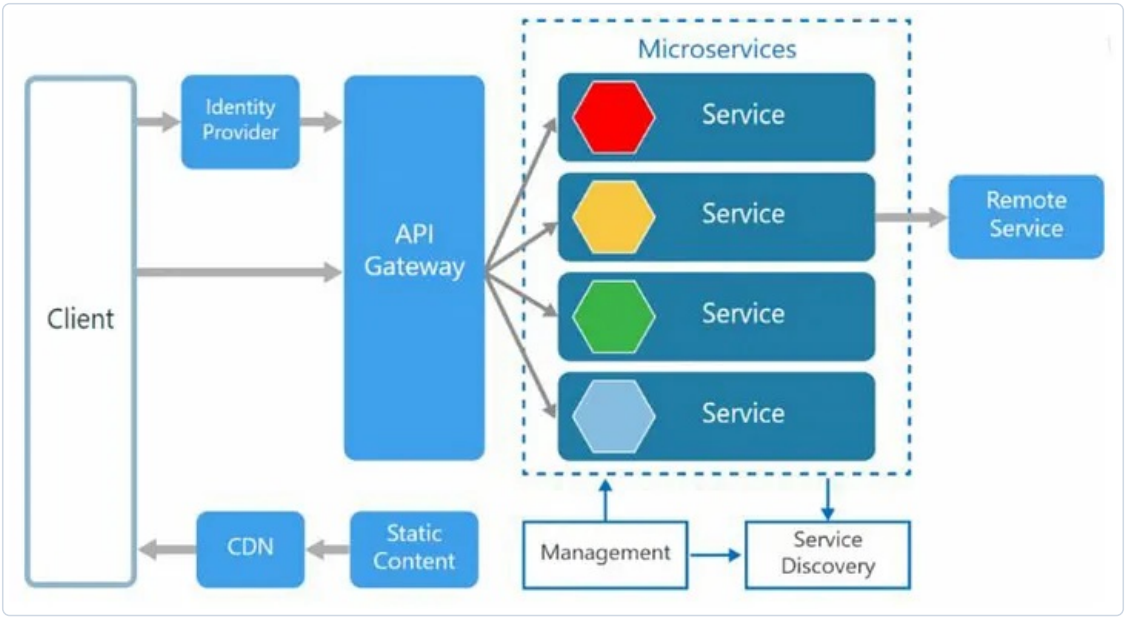


Fig. 4 — A more complete picture: the Client authenticates through an Identity Provider before reaching the API Gateway, static content is offloaded to a CDN, the API Gateway can reach a Remote Service outside the cluster, and a separate Management/Service Discovery pairing keeps track of which service instances are currently healthy and reachable.

This reference architecture — commonly seen in Microsoft's own microservices guidance — adds three production concerns the simpler diagrams above don't show explicitly:

- **Identity Provider** — authentication happens before the gateway, not duplicated inside every individual service (the centralized cross-cutting concern discussed in Part I's API Gateway section).
- **CDN + Static Content** — static assets are served directly from a CDN rather than routed through the microservices layer at all, keeping that layer focused purely on dynamic, business-logic requests.
- **Service Discovery** — the gateway (and other services) look up healthy service instances dynamically rather than relying on fixed addresses, exactly as built hands-on in Part I's Service Registry section (Bonus A).

> **The pattern across all four:** every one of these reference architectures is really the same small set of building blocks — gateway, services with their own data, and some form of inter-service communication — with additional production concerns (message brokers, identity, CDN, service discovery) layered on as scale and maturity demand. None of the later diagrams invalidate the earlier ones; they extend them.

Hands-On Deep Dive: Implementing Reference Architecture 3 (Fig. 3)

Turning Fig. 3 — API Gateway + four services + a message broker — into running code takes surprisingly little configuration on top of what Parts I and III already build. Here's the gateway side wired up exactly to that shape.

application.yml (api-gateway) — one route per box in Fig. 3

```
spring:
  cloud:
    gateway:
      routes:
        - id: catalog
          uri: lb://catalog-service
          predicates: [ Path=/v1/products/** ]
        - id: shopping-cart
          uri: lb://shopping-cart-service
          predicates: [ Path=/v1/baskets/** ]
        - id: discount
          uri: lb://discount-service
          predicates: [ Path=/v1/discounts/** ]
        - id: ordering
          uri: lb://ordering-service
          predicates: [ Path=/v1/orders/** ]
```

Each service publishing to the shared broker in Fig. 3, using the same event contract style

```
@Configuration
public class MessageBrokerTopicsConfig {
    @Bean public NewTopic catalogEvents() { return TopicBuilder.name("catalog-events").partitions(6).replicas(1).build(); }
    @Bean public NewTopic cartEvents()    { return TopicBuilder.name("cart-events").partitions(6).replicas(1).build(); }
    @Bean public NewTopic discountEvents(){ return TopicBuilder.name("discount-events").partitions(6).replicas(1).build(); }
    @Bean public NewTopic orderEvents()   { return TopicBuilder.name("order-events").partitions(6).replicas(1).build(); }
}
```

Testing the Wiring Matches the Diagram

```
# confirm every route in Fig. 3 actually resolves through the gateway
curl -i http://localhost:8080/v1/products/123     # -> routes to catalog-service
curl -i http://localhost:8080/v1/baskets/cust-1    # -> routes to shopping-cart-service
curl -i http://localhost:8080/v1/discounts/SAVE10  # -> routes to discount-service
curl -i http://localhost:8080/v1/orders/456        # -> routes to ordering-service

# confirm every service's topic exists on the shared broker
docker exec -it kafka kafka-topics --bootstrap-server localhost:9092 --list
# expect: catalog-events, cart-events, discount-events, order-events
```

> **The takeaway from turning a diagram into config this quickly:** Fig. 3's box-and-arrow diagram and the YAML above are describing the exact same system — the diagram just leaves out the specific route predicates and topic names. Once you can read a reference architecture diagram fluently, translating it into Spring Cloud Gateway routes and Kafka topic declarations becomes close to mechanical.

5. Case Study: A Java Spring Boot Reference Implementation

Reference diagrams are useful, but seeing the same ideas as one coherent, fully-built system makes them concrete. This section walks through a complete Java/Spring Boot reference implementation of the e-commerce platform this guide series builds throughout Parts I–III — the same architecture other ecosystems implement with their own tooling (for example, .NET's eShopOnContainers reference app uses Ocelot in place of Spring Cloud Gateway), translated fully into the Spring stack this guide standardizes on.

The API Gateway / BFF Layer — Spring Cloud Gateway + Aggregator

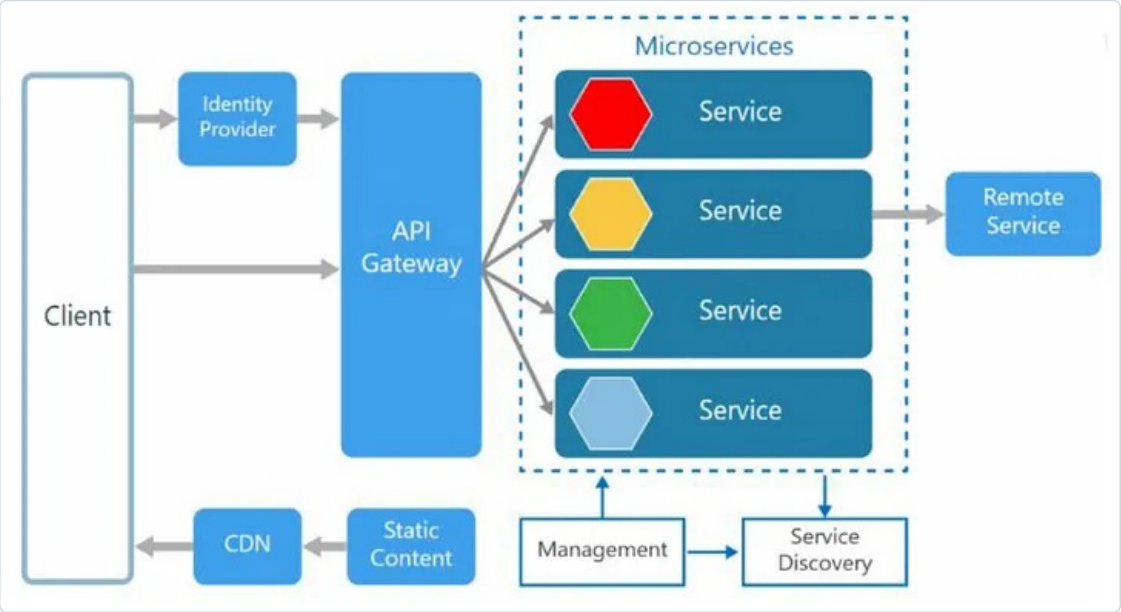


Fig. 5 — In Java/Spring Boot terms: the Client authenticates against an Identity Provider (a Spring Authorization Server or Keycloak instance) before reaching the API Gateway (Spring Cloud Gateway). The Gateway routes to four Spring Boot Services, can reach a Remote Service outside the cluster, and relies on a Management/Service Discovery pairing (Spring Boot Actuator + Eureka or Kubernetes-native discovery) to route only to healthy instances.

Mapped onto this guide's own Part I and Part III patterns:

- **Spring Cloud Gateway** is this stack's API Gateway (Part I, Pattern 6) — the direct Java equivalent of Ocelot in the .NET world, providing the same routing, aggregation, and cross-cutting-concerns feature set (including the Ocelot Features table used earlier purely as a cross-ecosystem comparison).
- A dedicated **Aggregator service**, built as its own Spring Boot application, fans out to multiple backend APIs in parallel and combines their results — exactly the technique built hands-on in Part I's Service Aggregator section (Bonus A), kept as its own deployable rather than embedded inside the gateway.
- **Spring Cloud Gateway routes per-client-type** where needed (Mobile-BFF, Web-BFF), each a separate Spring Boot application, implementing the Backend for Frontend pattern (Part I, Pattern 5) rather than one gateway trying to serve every client's shape at once.
- Where a specific internal call is high-volume or latency-sensitive — a Discount lookup, for example — that one service exposes a **gRPC** endpoint via `grpc-server-spring-boot-starter` instead of REST, exactly as built hands-on in Part III, Section 5.
- Each backend service uses the Spring Data module suited to its own workload: **Spring Data MongoDB** for Catalog (flexible product schema), **Spring Data Redis** for the Shopping Cart (fast, ephemeral key-value access), and **Spring Data JPA** over PostgreSQL or SQL Server for Ordering — polyglot persistence (Part I, Pattern 1) in practice, not just in theory.

The Full Picture — Every Client, Every Service, One Event Bus

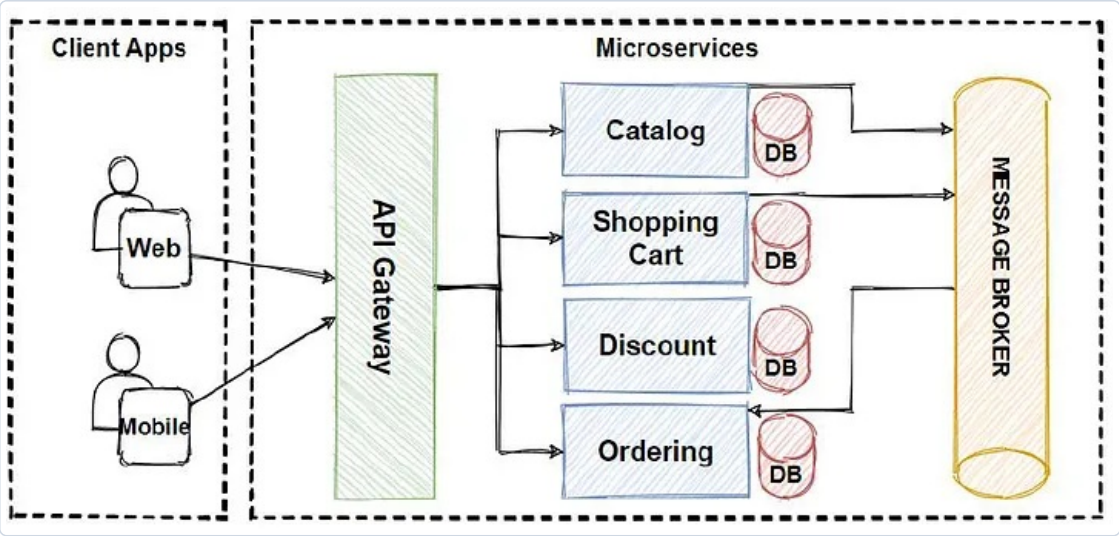


Fig. 6 — Web and Mobile clients enter through one Spring Cloud Gateway, which routes to Catalog, Shopping Cart, Discount, and Ordering — each its own Spring Boot application with its own database — all publishing to and consuming from a shared Kafka-backed Message Broker.

A few details worth calling out explicitly in the Java/Spring Boot version of this picture:

- **Multiple BFFs, split by both client type and business concern.** As client needs diverge, this guide's own pattern (Part I, Pattern 5) is to split further than just "Mobile" and "Web" — for example, a `mobile-shopping-bff` and a separate `mobile-marketing-bff` Spring Boot application, once a single per-client gateway starts accumulating too much unrelated logic.
- **Identity is its own Spring Boot microservice** — commonly built on Spring Authorization Server (or delegated to Keycloak), with its own relational database, participating in the same containerized deployment and service discovery as every other service rather than being a special-cased external system.
- **Apache Kafka is this guide's event bus** throughout Parts I–III, wired into every service via `spring-kafka` — playing exactly the role RabbitMQ or Azure Service Bus plays in other ecosystems' equivalent reference architectures. As covered in Part I's Bonus B, RabbitMQ remains a perfectly valid alternative broker choice; this guide standardizes on Kafka specifically because CQRS and Event Sourcing (Part I, Patterns 2–3; Part II throughout) depend on its replayable log.
- **Everything deploys as containers** — each Spring Boot service packaged with its own Dockerfile (or via Spring Boot's built-in Cloud Native Buildpacks support), orchestrated with Docker Compose locally and Kubernetes in production, exactly as set up in Part I, Section 2 and scaled in Part II, Section 10.

Minimal Project Layout for This Reference Implementation

A Maven multi-module layout matching Fig. 6, all Spring Boot

```
ecommerce-platform/
├── api-gateway/            # Spring Cloud Gateway — Part I, Pattern 6
├── catalog-service/        # Spring Boot + Spring Data MongoDB
├── shopping-cart-service/  # Spring Boot + Spring Data Redis
├── discount-service/       # Spring Boot + gRPC server — Part III, Section 5
├── ordering-service/       # Spring Boot + Spring Data JPA (PostgreSQL/SQL Server)
├── identity-service/       # Spring Boot + Spring Authorization Server
├── aggregator-service/     # Spring Boot — Part I, Bonus A
└── config-server/          # Spring Cloud Config — Part I, Pattern 9
```

Hands-On Deep Dive: The Three Pieces That Make Fig. 5 & 6 Real

api-gateway — the Mobile-Shopping / Web-Shopping split from Fig. 6

```
spring:
  cloud:
    gateway:
      routes:
        - id: mobile-shopping
          uri: lb://aggregator-service
          predicates: [ Path=/mobile/shopping/**, Header=X-Client-Type, mobile ]
        - id: web-shopping
          uri: lb://aggregator-service
          predicates: [ Path=/web/shopping/**, Header=X-Client-Type, web ]
        - id: identity
          uri: lb://identity-service
          predicates: [ Path=/auth/** ]
  security:
    oauth2:
      resourceserver:
        jwt:
          issuer-uri: http://identity-service:9000
```

aggregator-service — fanning out to Catalog, Basket, and Discount in one call (Part I, Bonus A)

```
@RestController
public class HomeAggregatorController {
    private final WebClient catalogClient;
    private final WebClient basketClient;
    private final WebClient discountClient;

    @GetMapping("/mobile/shopping/home/{customerId}")
    public Mono<MobileHomeView> home(@PathVariable String customerId) {
        Mono<List<ProductDto>> featured = catalogClient.get().uri("/v1/products?featured=true")
            .retrieve().bodyToFlux(ProductDto.class).collectList();
        Mono<BasketDto> basket = basketClient.get().uri("/v1/baskets/{id}", customerId)
            .retrieve().bodyToMono(BasketDto.class);
        Mono<List<DiscountDto>> activeDiscounts = discountClient.get().uri("/v1/discounts/active")
            .retrieve().bodyToFlux(DiscountDto.class).collectList();

        // all three calls fire concurrently -- this is the whole point of an Aggregator
        return Mono.zip(featured, basket, activeDiscounts)
            .map(t -> new MobileHomeView(t.getT1(), t.getT2(), t.getT3()));
    }
}
```

identity-service — Spring Authorization Server, issuing tokens the gateway validates above

```
@Configuration
public class AuthServerConfig {
    @Bean
    public RegisteredClientRepository registeredClientRepository() {
        RegisteredClient webApp = RegisteredClient.withId(UUID.randomUUID().toString())
            .clientId("web-shopping-client")
            .clientAuthenticationMethod(ClientAuthenticationMethod.CLIENT_SECRET_BASIC)
            .authorizationGrantType(AuthorizationGrantType.AUTHORIZATION_CODE)
            .redirectUri("https://shop.example.com/login/callback")
            .scope("orders.read").scope("orders.write")
            .build();
        return new InMemoryRegisteredClientRepository(webApp);
    }
}
```

Testing the Full Chain

```
# obtain a token from identity-service via the gateway
curl -X POST http://localhost:8080/auth/oauth2/token \
  -d "grant_type=client_credentials&client_id=web-shopping-client&client_secret=..."

# use it to call the aggregated mobile home screen through the gateway
curl http://localhost:8080/mobile/shopping/home/cust-1 \
  -H "Authorization: Bearer $TOKEN" -H "X-Client-Type: mobile"
# one call returns featured products + basket + active discounts, fetched in parallel by the aggregator
```

> **Why this reframing matters:** the architecture itself — API Gateway, BFF, polyglot persistence, an event bus, service discovery, centralized identity — is identical regardless of ecosystem. What changes between a .NET reference implementation and this one is purely the concrete tooling: Ocelot becomes Spring Cloud Gateway, ASP.NET Core services become Spring Boot services, and RabbitMQ/Azure Service Bus becomes Kafka. Every pattern covered across Parts I–III of this guide applies unchanged; only the library names on the dependency list are different.

6. Designing Our E-Commerce Microservices Architecture

Starting From the Functional Requirements

With the reference architectures from Section 4 as inspiration and the characteristics from Section 1 as guardrails, the starting design for this guide's own e-commerce system follows directly from its functional requirements: list and filter products, manage a shopping cart, apply discounts, check out and create orders, and view order history. Each of those capabilities becomes its own bounded context — and therefore its own microservice — rather than a module inside one shared application.

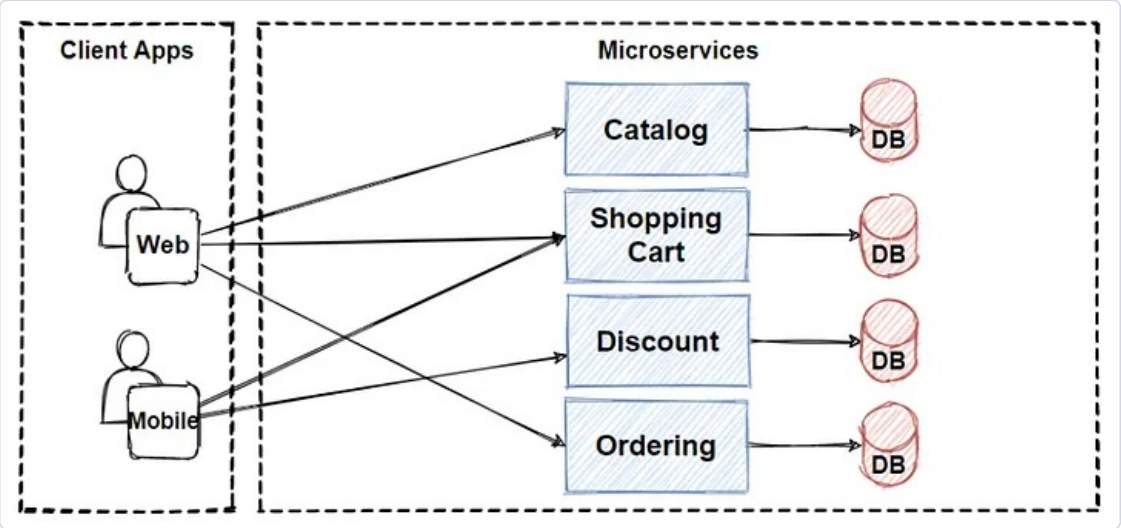


Fig. 7 — Web and Mobile clients call Catalog, Shopping Cart, Discount, and Ordering directly, each backed by its own independent database — Database per Microservice (Part I, Pattern 1) applied as the very first structural decision in this design, before any communication or data-sync pattern is layered on top.

Why Database per Microservice Comes First

Every other pattern in this guide series assumes this starting point. CQRS (Part I, Pattern 2) splits one service's database further into a write and read side — it doesn't make sense to reason about until each service already has its own database to split. The Saga pattern (Part I, Pattern 4) coordinates transactions *across* these independently-owned databases — it exists specifically because there's no shared database to lean on for a distributed ACID transaction. Getting this first structural decision right is what makes every later pattern in this guide a natural next step rather than a bolt-on afterthought.

Polyglot Persistence, Chosen Per Service

With each service now owning its own database, the technology choice for that database becomes a per-service decision rather than a platform-wide one:

Service	Data shape	A fitting database category
Catalog / Product	Flexible, evolving product attributes; read-heavy browsing	NoSQL document database (e.g., MongoDB)
Shopping Cart	Simple, ephemeral key-value data with a short lifespan	NoSQL key-value store (e.g., Redis)
Ordering	Strong consistency, relational integrity, financial correctness	Relational database (e.g., PostgreSQL, SQL Server)

This is the same reasoning applied to the Java Spring Boot reference implementation in Section 5 (MongoDB for Catalog, Redis for Basket, PostgreSQL/SQL Server for Ordering) — the database technology follows from the data's actual shape and access pattern, not from a single platform-wide default.

> **What handling FRs and NFRs together actually looks like here:** the functional requirements (list products, manage a cart, checkout) determine the service boundaries; the non-functional requirements — scalability and reliability under growing concurrent load, introduced back in Part I's capacity-planning table — determine which additional patterns (API Gateway, CQRS, Circuit Breaker, and so on) get layered on top of this basic structure as the system grows past what this simple design alone can comfortably handle.

Hands-On Deep Dive: The Three Services From Fig. 7, Fully Implemented

Catalog Service — MongoDB, read-heavy, flexible schema

```
@Document(collection = "products")
public class Product {
    @Id private String id;
    private String name;
    private String category;
    private BigDecimal price;
    private Map<String, Object> attributes;   // schema-flexible: size, color, brand, etc. per category
```

```
}

@RestController @RequestMapping("/v1/products")
public class ProductController {
    private final ProductRepository repository;

    @GetMapping public Page<Product> list(@RequestParam(required=false) String category, Pageable pageable) {
        return category != null ? repository.findByCategory(category, pageable) : repository.findAll(pageable);
    }
    @PostMapping @ResponseStatus(HttpStatus.CREATED)
    public Product create(@Valid @RequestBody Product product) { return repository.save(product); }
}
```

Shopping Cart Service — Redis, ephemeral key-value, fast reads/writes

```
@Service
public class BasketService {
    private final RedisTemplate<String, Basket> redisTemplate;
    private static final Duration TTL = Duration.ofDays(7); // abandoned carts expire automatically

    public Basket addItem(String customerId, String productId, int qty) {
        String key = "basket:" + customerId;
        Basket basket = redisTemplate.opsForValue().get(key);
        if (basket == null) basket = new Basket(customerId);
        basket.addLine(productId, qty);
        redisTemplate.opsForValue().set(key, basket, TTL);
        return basket;
    }
}
```

Ordering Service — PostgreSQL via JPA, strong consistency for checkout

```
@Entity @Table(name = "orders")
public class Order {
    @Id @GeneratedValue private UUID id;
    private String customerId;
    @Enumerated(EnumType.STRING) private OrderStatus status = OrderStatus.PENDING;
    private BigDecimal totalAmount;
}

@Service
public class OrderService {
    private final OrderRepository orderRepository;

    @Transactional
    public Order checkout(String customerId, List<OrderLine> lines) {
        BigDecimal total = lines.stream()
            .map(l -> l.unitPrice().multiply(BigDecimal.valueOf(l.quantity())))
            .reduce(BigDecimal.ZERO, BigDecimal::add);
        Order order = new Order(customerId, total, lines);
        return orderRepository.save(order); // ACID-committed -- this is why Ordering gets a relational DB
    }
}
```

Testing the Design End to End

```
# list products from the catalog (MongoDB)
curl http://localhost:8081/v1/products?category=Electronics

# add one to a basket (Redis, TTL-backed)
curl -X POST http://localhost:8082/v1/baskets/cust-1/items -d '{"productId":"p1","quantity":2}'

# checkout, creating an order (PostgreSQL, ACID)
curl -X POST http://localhost:8084/v1/orders/checkout \
  -d '{"customerId":"cust-1","lines":[{"productId":"p1","quantity":2,"unitPrice":19.99}]}'
```

7. Technology Stack Choices

Microservices Are a Polyglot Environment by Design

Because each service is independently deployable and communicates only through well-defined APIs (REST, gRPC, or asynchronous messaging — the full spectrum covered in Part III), there's no architectural requirement that every service share the same language, framework, or database. Each team can pick the stack best suited to their specific service's needs.

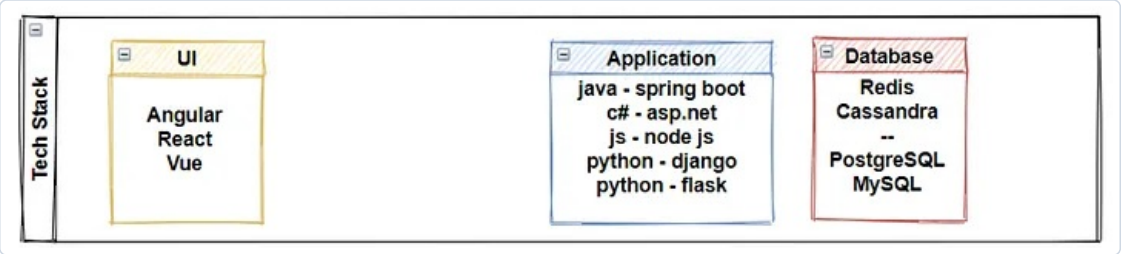


Fig. 8 — A concrete polyglot tech stack laid out by layer: UI options (Angular, React, Vue), Application/backend options (Java-Spring Boot, C#-ASP.NET, JS-Node.js, Python-Django, Python-Flask), and Database options (Redis, Cassandra, PostgreSQL, MySQL).

Layer	Representative options	Typical fit
UI / Frontend	Angular, React, Vue	Whichever the frontend team already has the strongest expertise in — largely decoupled from backend choices entirely
Backend / Application	Java + Spring Boot, C# + ASP.NET, JavaScript/TypeScript + Node.js, Python + Django, Python + Flask	Team familiarity, existing library ecosystem, and performance characteristics needed by that specific service
Database — NoSQL	MongoDB, Redis, Cassandra	Document flexibility (MongoDB), fast key-value access (Redis), high write-throughput at scale (Cassandra) — see Part I, Pattern 1 and Part II, Sections 2–5 for hands-on use of Cassandra specifically
Database — Relational	PostgreSQL, MySQL, Oracle, SQL Server	Strong consistency and relational integrity for transactional, financially-sensitive data — used throughout Part I and Part II for the Ordering write side

This Guide's Own Stack Choice, and Why

Parts I–III of this guide standardize on Java and Spring Boot specifically because it's a widely understood, strongly-typed, production-proven stack with first-class support for every pattern covered — Spring Cloud Gateway for the API Gateway pattern, Spring Kafka for event-driven communication, Spring Data for polyglot persistence across MongoDB/Cassandra/PostgreSQL/SQL Server, and Resilience4j for Circuit Breaking. None of that is a claim that Java/Spring Boot is the "correct" choice in some universal sense — per the polyglot principle above, a team building the same system in C#/ASP.NET or Python/Django would apply the exact same patterns, largely translated one-for-one (as Section 5's eShopOnContainers case study demonstrates for the .NET ecosystem).

> **A practical decision process for a new service:**
>
> 1. What does this service's data actually look like, and how is it queried? (Section 6's table above)
> 2. What does the team building this specific service already know well?
> 3. Does this service need to interoperate tightly with an existing internal library or platform convention that narrows the realistic options?
> 4. Only after those three — consider raw performance characteristics, since for most services, the language/framework choice matters far less than getting the data model and communication pattern right.

8. Resilience Patterns: Circuit Breaker, Retry, Rate Limiter & Bulkhead

Why This Section Exists

Section 3's Network Problems and Latency challenge is the root cause every pattern in this section addresses. Once a call crosses the network, it can fail in ways an in-process call never does: the callee can be slow, temporarily down, overwhelmed, or simply unreachable. Part I's Pattern 8 introduced the Circuit Breaker on its own; this section goes further and builds the **full resilience stack** that production Spring Boot microservices actually run — Circuit Breaker, Retry with exponential backoff, Rate Limiter, and Bulkhead, combined on the same call, each solving a distinct failure mode.

Pattern	Question it answers	Failure mode it protects against
Circuit Breaker	"Has this dependency failed enough that I should stop calling it?"	Cascading failure from a persistently broken dependency
Retry (+ Exponential Backoff)	"Was this failure transient enough to be worth trying again?"	Brief network blips, momentary unavailability
Rate Limiter	"Am I about to send more traffic than this dependency (or I) can handle?"	Self-inflicted overload, protecting both caller and callee
Bulkhead	"Can a slow dependency exhaust all of my threads/connections?"	Resource exhaustion spreading from one slow dependency to unrelated calls

Shared Setup

pom.xml

```
<dependency>
  <groupId>io.github.resilience4j</groupId>
  <artifactId>resilience4j-spring-boot3</artifactId>
  <version>2.2.0</version>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-aop</artifactId>
</dependency>
```

9.1 Circuit Breaker — Stopping Calls to a Failing Dependency

The three-state model (CLOSED → OPEN → HALF_OPEN) is covered in depth in Part I, Pattern 8. Here it's the outermost layer wrapping every other pattern in this section — once the breaker opens, none of Retry, Rate Limiter, or Bulkhead below it even get a chance to run, because the call fails fast before reaching them.

application.yml

```
resilience4j:
  circuitbreaker:
    instances:
      discountService:
        slidingWindowSize: 20
        minimumNumberOfCalls: 10
        failureRateThreshold: 50
        waitDurationInOpenState: 15000
        permittedNumberOfCallsInHalfOpenState: 3
```

9.2 Retry with Exponential Backoff — Giving Transient Failures a Second Chance

Not every failure deserves a circuit breaker's suspicion — a single dropped packet or a momentary GC pause on the other side is often gone by the time you'd retry. Retry handles exactly this case, and **exponential backoff** is what keeps a retry storm from making a struggling dependency's day worse: each successive attempt waits longer than the last, spacing out load instead of hammering a recovering service with immediate retries from every caller at once.

application.yml — exponential backoff: 200ms, then 400ms, then 800ms

```
resilience4j:
  retry:
    instances:
      discountService:
        maxAttempts: 4
        waitDuration: 200
        enableExponentialBackoff: true
        exponentialBackoffMultiplier: 2
```

```
        retryExceptions:
          - java.io.IOException
          - org.springframework.web.client.ResourceAccessException
        ignoreExceptions:
          - com.example.discount.DiscountNotFoundException   # a 404 is not transient -- don't retry it
```

> **Why exponential, not fixed-interval, backoff:** with 50 callers all retrying a struggling service every fixed 200ms, that service now faces the same overload it was already struggling with, five times over. Exponential backoff (200ms → 400ms → 800ms → 1600ms...) naturally spreads retries out over time as attempts increase, and pairing it with *jitter* (a small random offset added to each wait) avoids callers that started at the same instant retrying in lockstep with each other.

application.yml — the same idea, with jitter added

```
resilience4j:
  retry:
    instances:
      discountService:
        maxAttempts: 4
        waitDuration: 200
        enableExponentialBackoff: true
        exponentialBackoffMultiplier: 2
        enableRandomizedWait: true
        randomizedWaitFactor: 0.5   # actual wait = base wait +/- up to 50%, breaking lockstep retries
```

9.3 Rate Limiter — Not Sending More Than the Dependency (or You) Can Handle

A Rate Limiter caps how many calls are allowed through in a given time window — protecting a downstream dependency from being overwhelmed, and just as often protecting the calling service itself from exceeding a third-party API's contractual quota. Section 10 covers the algorithms behind rate limiting in depth; here is the minimal Resilience4j wiring.

application.yml

```
resilience4j:
  ratelimiter:
    instances:
      discountService:
        limitForPeriod: 50        # 50 calls allowed...
        limitRefreshPeriod: 1s     # ...per 1-second window
        timeoutDuration: 0         # don't wait for a slot -- fail immediately if the limit is hit
```

9.4 Bulkhead — Containing a Slow Dependency's Blast Radius

Named after a ship's watertight compartments, a Bulkhead caps how many concurrent calls to one dependency are allowed at once — so a slow Discount service can only ever tie up a bounded number of threads/connections, leaving the rest of the calling service's capacity free to keep serving requests that don't depend on Discount at all.

application.yml — thread-pool bulkhead, isolating Discount calls onto their own executor

```
resilience4j:
  thread-pool-bulkhead:
    instances:
      discountService:
        maxThreadPoolSize: 10
        coreThreadPoolSize: 5
        queueCapacity: 20
```

9.5 Combining All Four on One Call

Stacked together, annotation order matters: Resilience4j applies decorators outside-in in the order listed, so the outermost annotation is evaluated first. The recommended order for this stack is Bulkhead → Rate Limiter → Circuit Breaker → Retry, so that a request is first admitted by capacity controls (Bulkhead, Rate Limiter) before the Circuit Breaker's failure tracking and Retry's re-attempts ever get involved.

DiscountClient.java — all four resilience patterns on one method

```
@Service
public class DiscountClient {
    private final RestClient restClient;
    private static final Logger log = LoggerFactory.getLogger(DiscountClient.class);

    @Bulkhead(name = "discountService", type = Bulkhead.Type.THREADPOOL)
    @RateLimiter(name = "discountService")
    @CircuitBreaker(name = "discountService", fallbackMethod = "fallback")
    @Retry(name = "discountService")
    public CompletableFuture<DiscountDto> getDiscount(String code) {
        return CompletableFuture.supplyAsync(() ->
            restClient.get().uri("http://discount-service/v1/discounts/{code}", code)
                .retrieve().body(DiscountDto.class));
```

```
    }

    public CompletableFuture<DiscountDto> fallback(String code, Throwable t) {
        log.warn("Discount service degraded for code={}, serving no-discount fallback: {}", code, t.toString());
        return CompletableFuture.completedFuture(DiscountDto.none(code));
    }
}
```

Testing the Full Stack

```
# 1. Bulkhead: fire more concurrent requests than maxThreadPoolSize+queueCapacity allows
for i in $(seq 1 40); do curl -s -o /dev/null -w "%{http_code}\n" http://localhost:8082/orders/with-discount/SAVE10 & done
# expect some requests to fail fast with a BulkheadFullException-derived response once capacity is exceeded

# 2. Rate Limiter: exceed limitForPeriod within one window
for i in $(seq 1 60); do curl -s -o /dev/null -w "%{http_code} " http://localhost:8082/orders/with-discount/SAVE10; done
# expect ~50 successes then fast failures for the remainder of that 1-second window

# 3. Circuit Breaker + Retry together: stop discount-service, then hit the endpoint
docker stop discount-service
curl -w "\ntime: %{time_total}s\n" http://localhost:8082/orders/with-discount/SAVE10
# first several calls: visibly slower (Retry backing off through 200/400/800ms attempts)
# after failureRateThreshold crossed: instant fallback response (breaker OPEN, Retry never even engages)
```

Gotchas:

- Retry and Circuit Breaker can fight each other if misconfigured — make sure `minimumNumberOfCalls` for the breaker accounts for the fact that Retry means each logical call can generate several physical attempts; count accordingly or the breaker trips far earlier than intended.
- A Bulkhead's `queueCapacity` adds latency for queued calls even when they eventually succeed — size it based on acceptable added wait time, not just raw throughput.
- Never retry a non-idempotent write without a separate idempotency key (Part I, Bonus D's Outbox Pattern discussion covers why) — blind retries on `POST /orders` without one can create duplicate orders.

9. Rate Limiter Algorithms — A Practical Approach

Why the Algorithm Choice Matters

Section 9.3 wired up a rate limiter with two numbers — `limitForPeriod` and `limitRefreshPeriod` — without asking how that limit is actually enforced moment to moment. That enforcement detail is exactly where rate limiter algorithms differ, and the choice has real consequences: some algorithms allow bursty traffic right at window boundaries, some smooth traffic out perfectly but cost more memory, and some are trivial to implement but let through up to 2x your intended limit. This section builds each major algorithm from scratch in Java so the trade-offs are concrete, not just theoretical.

10.1 Fixed Window Counter

The simplest possible approach: count requests in a fixed time window (e.g., "this minute"), reset the counter to zero the instant a new window starts.

FixedWindowRateLimiter.java

```
public class FixedWindowRateLimiter {
    private final int maxRequests;
    private final long windowMillis;
    private AtomicInteger counter = new AtomicInteger(0);
    private volatile long windowStart = System.currentTimeMillis();

    public synchronized boolean allowRequest() {
        long now = System.currentTimeMillis();
        if (now - windowStart >= windowMillis) {
            windowStart = now;       // new window -- counter resets to zero, all at once
            counter.set(0);
        }
        return counter.incrementAndGet() <= maxRequests;
    }
}
```

The boundary burst problem: with a 100-req/minute limit, a client can send 100 requests at `00:00:59` and another 100 at `00:01:00` — 200 requests in a two-second span, even though the average rate never exceeds the stated limit within any single fixed window. Simple to implement, cheap in memory, but this burst behavior is often not what "100 requests per minute" was actually meant to guarantee.

10.2 Sliding Window Log

Fixes the boundary-burst problem by tracking the exact timestamp of every request in a log, and counting how many fall within the trailing window — not a fixed calendar window, but a window that slides continuously with the current time.

SlidingWindowLogRateLimiter.java

```
public class SlidingWindowLogRateLimiter {
    private final int maxRequests;
    private final long windowMillis;
    private final Deque<Long> requestLog = new ConcurrentLinkedDeque<>();

    public synchronized boolean allowRequest() {
        long now = System.currentTimeMillis();
        long windowStart = now - windowMillis;

        // drop every timestamp that has aged out of the trailing window
        while (!requestLog.isEmpty() && requestLog.peekFirst() < windowStart) {
            requestLog.pollFirst();
        }

        if (requestLog.size() < maxRequests) {
            requestLog.offerLast(now);
            return true;
        }
        return false;
    }
}
```

The trade-off: perfectly accurate — no boundary bursts, ever — but memory cost is proportional to the request rate, since every single request's timestamp is retained until it ages out. High-traffic endpoints with a generous limit can mean tracking thousands of timestamps per client at once.

10.3 Sliding Window Counter — The Practical Middle Ground

Approximates the sliding window log's accuracy at a fraction of the memory cost, by keeping just two fixed-window counters (the current window and the previous one) and computing a weighted estimate of how many requests fall in the trailing window based on how far into the current window "now" is.

SlidingWindowCounterRateLimiter.java

```
public class SlidingWindowCounterRateLimiter {
    private final int maxRequests;
    private final long windowMillis;
    private volatile long currentWindowStart = System.currentTimeMillis();
    private AtomicInteger currentCount = new AtomicInteger(0);
    private AtomicInteger previousCount = new AtomicInteger(0);

    public synchronized boolean allowRequest() {
        long now = System.currentTimeMillis();
        if (now - currentWindowStart >= windowMillis) {
            previousCount.set(currentCount.get());   // this window becomes "previous" for the next check
            currentCount.set(0);
            currentWindowStart = now;
        }

        double elapsedFraction = (now - currentWindowStart) / (double) windowMillis;
        // weight the previous window's count by how much of the trailing window still overlaps it
        double estimatedCount = previousCount.get() * (1 - elapsedFraction) + currentCount.get();

        if (estimatedCount < maxRequests) {
            currentCount.incrementAndGet();
            return true;
        }
        return false;
    }
}
```

Why this is the industry-default choice: constant memory (two counters, regardless of traffic volume), and the boundary-burst problem is reduced to a small approximation error rather than eliminated only at high memory cost. Cloudflare and several major API gateway products use exactly this algorithm for this reason — it's the practical sweet spot between Fixed Window's simplicity and Sliding Window Log's precision.

10.4 Token Bucket

A bucket holds up to `capacity` tokens; tokens refill continuously at a fixed rate; every request consumes one token; a request is rejected only if the bucket is empty. Unlike the window-based algorithms above, Token Bucket naturally allows short bursts up to the bucket's full capacity, then throttles down to the steady refill rate — often a better fit for real client behavior (a user clicking rapidly, then pausing) than a hard per-window cap.

TokenBucketRateLimiter.java

```
public class TokenBucketRateLimiter {
    private final double capacity;
    private final double refillTokensPerMillis;
    private double availableTokens;
    private long lastRefillTimestamp = System.currentTimeMillis();

    public TokenBucketRateLimiter(double capacity, double refillTokensPerSecond) {
        this.capacity = capacity;
        this.availableTokens = capacity;   // start full -- allows an initial burst
        this.refillTokensPerMillis = refillTokensPerSecond / 1000.0;
    }

    public synchronized boolean allowRequest() {
        refill();
        if (availableTokens >= 1) {
            availableTokens -= 1;
            return true;
        }
        return false;
    }

    private void refill() {
        long now = System.currentTimeMillis();
        double tokensToAdd = (now - lastRefillTimestamp) * refillTokensPerMillis;
        availableTokens = Math.min(capacity, availableTokens + tokensToAdd);
        lastRefillTimestamp = now;
    }
}
```

10.5 Leaky Bucket

The inverse mental model of Token Bucket: incoming requests fill a bucket of fixed capacity; a background process drains ("leaks") the bucket at a constant rate, processing one request at a time; if the bucket is full, new requests are rejected outright. Where Token Bucket allows bursts through immediately, Leaky Bucket smooths all traffic to a strictly constant output rate — the trade-off is added latency for queued requests versus Token Bucket's immediate pass-through up to capacity.

LeakyBucketRateLimiter.java

```
public class LeakyBucketRateLimiter {
    private final Queue<Runnable> bucket = new ConcurrentLinkedQueue<>();
```

```
    private final int capacity;

    public LeakyBucketRateLimiter(int capacity, long leakIntervalMillis) {
        this.capacity = capacity;
        // a single-threaded scheduler drains exactly one request per tick -- the constant "leak" rate
        Executors.newSingleThreadScheduledExecutor().scheduleAtFixedRate(() -> {
            Runnable task = bucket.poll();
            if (task != null) task.run();
        }, 0, leakIntervalMillis, TimeUnit.MILLISECONDS);
    }

    public boolean submit(Runnable request) {
        if (bucket.size() >= capacity) return false;  // bucket full -- reject immediately
        bucket.offer(request);
        return true;
    }
}
```

Comparing All Five Algorithms

Algorithm	Allows bursts?	Memory cost	Boundary accuracy	Best fit
Fixed Window Counter	Yes (at boundaries, up to 2x limit)	Constant, minimal	Poor	Simple, low-stakes limits where occasional boundary bursts are acceptable
Sliding Window Log	No	Proportional to request volume	Perfect	Low-to-moderate traffic where exact accuracy matters more than memory
Sliding Window Counter	Minimal, bounded approximation error	Constant, minimal	Very good	The default choice for most production API rate limiting
Token Bucket	Yes, up to bucket capacity, by design	Constant, minimal	Good (burst-tolerant by design, not by accident)	Client-facing APIs where legitimate bursty usage patterns are expected (e.g., a user rapidly paging through results)
Leaky Bucket	No — output is a strictly constant rate	Constant, minimal	Perfect for output rate	Protecting a downstream system that genuinely cannot handle any burst at all (e.g., a legacy system with a hard concurrency ceiling)

Which Algorithm Does Resilience4j Actually Use?

Resilience4j's `RateLimiter` (used in Section 9.3) implements a variant close to the Fixed Window Counter model — a semaphore-based limiter that refreshes permits every `limitRefreshPeriod`. For workloads where the boundary-burst behavior matters, two practical upgrades are common in production Spring Boot services:

- **Bucket4j** — a dedicated Java rate-limiting library implementing true Token Bucket semantics, with a `bucket4j-spring-boot-starter` integration and, notably, a Redis-backed distributed mode.
- **A Redis-backed Sliding Window Counter** — necessary the moment rate limiting needs to be enforced consistently *across multiple instances* of a service, since every in-memory implementation above only limits traffic hitting that one JVM.

Distributed rate limiting with Bucket4j + Redis — the production upgrade path

```
<dependency>
  <groupId>com.bucket4j</groupId>
  <artifactId>bucket4j-redis</artifactId>
  <version>8.10.1</version>
</dependency>
```

```
@Service
public class DistributedRateLimiter {
    private final ProxyManager<String> proxyManager;   // backed by a shared Redis instance

    public boolean tryConsume(String clientId) {
        BucketConfiguration config = BucketConfiguration.builder()
            .addLimit(Bandwidth.classic(100, Refill.intervally(100, Duration.ofMinutes(1))))
            .build();
        Bucket bucket = proxyManager.builder().build(clientId, () -> config);
        return bucket.tryConsume(1);
        // every instance of this service checks the SAME Redis-backed bucket for clientId --
        // the limit is enforced platform-wide, not per-JVM
    }
}
```

> **The single most common production mistake with rate limiting:** implementing any of the in-memory algorithms above and deploying the service with more than one replica. Each instance enforces its own independent limit, so a client hitting a 100-req/minute-limited service running on 5 replicas can actually push up to 500 requests per minute through, simply by getting load-balanced across all five. Any limit that must hold platform-wide, not per-instance, needs a shared store (Redis, as above) behind it.

> **Decision guide:** reach for Fixed Window only for coarse, forgiving limits; reach for Sliding Window Counter as the sensible default for most API

rate limiting; reach for Token Bucket specifically when bursty client behavior is expected and desired; reach for Leaky Bucket when the downstream system genuinely cannot tolerate any burst at all; and move any of these to a Redis-backed (or equivalent shared-store) implementation the moment the service runs more than one replica.

10. Caching Strategies: Caffeine & Redis with Eviction Policies

Why Caching Belongs Right After Rate Limiting

Section 9's rate limiters protect a dependency by rejecting excess requests. Caching takes a different approach to the same underlying problem — Section 3's Network Problems and Latency challenge — by avoiding the call to the dependency altogether whenever the answer hasn't changed since it was last fetched. The two techniques compose well together: caching reduces how often you need to call a rate-limited dependency in the first place, directly lowering how often that rate limit is even a concern.

This section covers caching at two levels that are almost always used together in a real Spring Boot microservice: **Caffeine**, an in-process (per-JVM) cache with no network hop at all, and **Redis**, a shared, distributed cache every instance of a service can see consistently. Both need an eviction policy — a rule for what gets removed when the cache is full or data goes stale — and this section builds all three major policies (LRU, LFU, TTL) hands-on, on both technologies.

Aspect	Caffeine (local)	Redis (distributed)
Location	In-process heap memory, one copy per service instance	A separate, shared process every instance of the service (and other services) can read/write
Latency	Nanoseconds — no network hop	Sub-millisecond — one network round trip
Consistency across instances	None — each instance's cache can disagree with another's	Shared — every instance sees the same cached value
Capacity	Bounded by that one JVM's heap	Bounded by the Redis instance/cluster's memory, shared across all consumers
Best fit	Extremely hot, small data (e.g., a handful of feature flags, a discount-code lookup table)	Larger or must-be-consistent data (e.g., product catalog entries, session data)

10.1 Eviction Policy: LRU (Least Recently Used)

LRU evicts whichever cached entry hasn't been *accessed* for the longest time — the intuition being that data nobody has asked for recently is the least likely to be asked for again soon. It's the most common default eviction policy across caching libraries.

Conceptual LRU cache from scratch (what Caffeine implements internally, simplified)

```
public class SimpleLruCache<K, V> {
    private final int capacity;
    private final LinkedHashMap<K, V> store;

    public SimpleLruCache(int capacity) {
        this.capacity = capacity;
        // accessOrder=true reorders entries on every get(), not just put() -- this IS the LRU mechanism
        this.store = new LinkedHashMap<>(16, 0.75f, true) {
            @Override
            protected boolean removeEldestEntry(Map.Entry<K, V> eldest) {
                return size() > capacity;   // evict the least-recently-accessed entry once over capacity
            }
        };
    }

    public synchronized V get(K key) { return store.get(key); }
    public synchronized void put(K key, V value) { store.put(key, value); }
}
```

10.2 Eviction Policy: LFU (Least Frequently Used)

LFU evicts whichever entry has been accessed the *fewest total times*, regardless of how recently — better suited than LRU when some entries are consistently popular over the long run but happen to have a brief lull, which LRU would incorrectly evict.

Conceptual LFU cache from scratch

```
public class SimpleLfuCache<K, V> {
    private final int capacity;
    private final Map<K, V> store = new HashMap<>();
    private final Map<K, Integer> frequency = new HashMap<>();

    public synchronized V get(K key) {
        if (store.containsKey(key)) frequency.merge(key, 1, Integer::sum);
        return store.get(key);
    }

    public synchronized void put(K key, V value) {
        if (!store.containsKey(key) && store.size() >= capacity) {
            K leastUsed = frequency.entrySet().stream()
                .min(Map.Entry.comparingByValue())
                .map(Map.Entry::getKey).orElseThrow();
            store.remove(leastUsed);
```

```
            frequency.remove(leastUsed);
        }
        store.put(key, value);
        frequency.merge(key, 1, Integer::sum);
    }
}
```

10.3 Eviction Policy: TTL (Time to Live)

TTL evicts an entry once a fixed amount of time has passed, regardless of how often or recently it was accessed — the right choice whenever an entry's *correctness* degrades over time, independent of its popularity (a stock price quote, an exchange rate, a session token).

> **These three policies aren't mutually exclusive.** Real caches routinely combine them: TTL caps how long any entry can live regardless of popularity (correctness), while LRU or LFU decides which entries get evicted early to make room, before their TTL would have expired anyway (capacity). Both Caffeine and Redis, built hands-on below, support exactly this combination.

Hands-On: Caffeine — Local, In-Process Caching

pom.xml

```
<dependency>
  <groupId>com.github.ben-manes.caffeine</groupId>
  <artifactId>caffeine</artifactId>
  <version>3.1.8</version>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-cache</artifactId>
</dependency>
```

CaffeineCacheConfig.java — LRU-by-size, TTL, and a recording stats window, all on one cache

```
@Configuration
@EnableCaching
public class CaffeineCacheConfig {

    @Bean
    public CacheManager cacheManager() {
        CaffeineCacheManager manager = new CaffeineCacheManager("discounts", "products");
        manager.setCaffeine(Caffeine.newBuilder()
            .maximumSize(10_000)                          // LRU-style eviction once this size is exceeded
            .expireAfterWrite(Duration.ofMinutes(10))       // TTL: correctness ceiling regardless of popularity
            .expireAfterAccess(Duration.ofMinutes(2))       // idle entries die faster than their write-TTL
            .recordStats());                                // exposes hit/miss/eviction metrics via Micrometer
        return manager;
    }
}
```

DiscountService.java — declarative caching with Spring's @Cacheable

```
@Service
public class DiscountService {
    private final DiscountRepository repository;

    @Cacheable(cacheNames = "discounts", key = "#code")
    public Discount getByCode(String code) {
        // this method body only runs on a cache MISS -- a hit returns straight from Caffeine, no DB call
        return repository.findByCode(code).orElseThrow(() -> new DiscountNotFoundException(code));
    }

    @CacheEvict(cacheNames = "discounts", key = "#discount.code")
    public Discount update(Discount discount) {
        return repository.save(discount);   // explicit invalidation the instant the source of truth changes
    }
}
```

Configuring LFU-Style Behavior in Caffeine

Caffeine's default `maximumSize` eviction uses a frequency-and-recency-aware algorithm (a variant of LFU called TinyLFU, layered with an LRU-like recency window) rather than pure LRU — in practice this means Caffeine already approximates the "protect popular entries from a brief lull" benefit LFU provides, without any extra configuration beyond `maximumSize`.

Hands-On: Redis — Shared, Distributed Caching

pom.xml

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-redis</artifactId>
</dependency>
```

application.yml

```
spring:
  cache:
    type: redis
  data:
    redis:
      host: localhost
      port: 6379
```

RedisCacheConfig.java — TTL per cache name

```
@Configuration
@EnableCaching
public class RedisCacheConfig {
    @Bean
    public RedisCacheManagerBuilderCustomizer redisCacheManagerBuilderCustomizer() {
        return builder -> builder
            .withCacheConfiguration("products",
                RedisCacheConfiguration.defaultCacheConfig().entryTtl(Duration.ofMinutes(30)))
            .withCacheConfiguration("sessions",
                RedisCacheConfiguration.defaultCacheConfig().entryTtl(Duration.ofHours(2)));
    }
}
```

redis.conf — configuring Redis's OWN eviction policy for when memory fills up

```
maxmemory 512mb
maxmemory-policy allkeys-lru   # evict least-recently-used keys across the whole keyspace once full
# alternatives: allkeys-lfu (least-frequently-used), volatile-ttl (evict soonest-to-expire keys first,
# only among keys that HAVE a TTL set), noeviction (reject writes instead of evicting -- rarely what you want)
```

> **Two eviction layers, working together:** Spring's `entryTtl` above proactively expires entries on a schedule (correctness); Redis's `maxmemory-policy` is a safety net that only kicks in if the cache fills up before TTL expiry does the job (capacity) — exactly the "TTL for correctness, LRU/LFU for capacity" combination introduced in Section 10.3.

Combining Both: A Two-Tier Cache (Caffeine L1 + Redis L2)

A common production pattern checks the fast, local Caffeine cache first; on a miss, checks the shared Redis cache next; only on a miss in *both* does the actual database get called — and a Redis hit populates Caffeine so the next request on that same instance is served purely from memory.

TwoTierProductCache.java

```
@Service
public class TwoTierProductCache {
    private final Cache<String, Product> l1Cache;     // Caffeine
    private final RedisTemplate<String, Product> l2Cache; // Redis
    private final ProductRepository repository;             // MongoDB, the source of truth

    public Product getProduct(String id) {
        Product cached = l1Cache.getIfPresent(id);
        if (cached != null) return cached;                       // L1 hit -- nanoseconds

        Product fromRedis = l2Cache.opsForValue().get("product:" + id);
        if (fromRedis != null) {
            l1Cache.put(id, fromRedis);                              // promote to L1 for next time
            return fromRedis;                                         // L2 hit -- one network hop
        }

        Product fromDb = repository.findById(id).orElseThrow(() -> new ProductNotFoundException(id));
        l2Cache.opsForValue().set("product:" + id, fromDb, Duration.ofMinutes(30));
        l1Cache.put(id, fromDb);
        return fromDb;                                               // full miss -- database hit, now cached at both tiers
    }
}
```

Testing Eviction Behavior Directly

```
# Caffeine: confirm hit/miss/eviction counts via Actuator + Micrometer
curl http://localhost:8083/actuator/metrics/cache.gets?tag=cache:discounts
curl http://localhost:8083/actuator/metrics/cache.evictions?tag=cache:discounts

# Redis: watch a key's remaining TTL count down
```

```
redis-cli SET product:123 '{"name":"Wireless Mouse"}' EX 1800
redis-cli TTL product:123        # -> 1800, then decreasing on subsequent calls

# Redis: confirm the configured eviction policy under memory pressure
redis-cli CONFIG GET maxmemory-policy
redis-cli INFO stats | grep evicted_keys   # non-zero once maxmemory is actually reached
```

Gotchas:

- **Cache stampede:** if a hot key expires and 1,000 concurrent requests all miss at once, they can all hit the database simultaneously. Guard against this with a short-lived lock/single-flight pattern (only one request repopulates the cache; the rest wait briefly and read the freshly-populated value) or by staggering TTLs with a small random jitter so not every instance's cache expires at the exact same instant.
- **Stale reads after a write:** `@CacheEvict` must fire in the same logical operation as the underlying write (as shown in `DiscountService.update()` above) — forgetting it means the cache keeps serving pre-update data for the rest of its TTL.
- **Caffeine caches are per-instance:** two replicas of the same service can legitimately disagree for a short window after an update, since each has its own independent L1 cache — this is why L2 (Redis) exists for data that must be consistent across instances, and why Caffeine alone is best reserved for data that's fine being briefly, locally stale.

Decision guide: use Caffeine alone for small, extremely hot, tolerant-of-brief-staleness data local to one instance. Use Redis alone when data must be consistent across every instance or is too large for comfortable per-instance heap usage. Use the two-tier combination when a value is both hot enough to benefit from zero-network-hop reads *and* needs to stay consistent across instances after a Redis-level update — Redis remains the source of truth for the cache layer, Caffeine is purely a short-lived, best-effort accelerator in front of it.

11. Docker & Kubernetes: Architecture, Components & Core Operations

Every service built across this guide series has assumed containers and orchestration were already running underneath it — Part I's Docker Compose stack, Part II's Kubernetes HPA configs, Part IV's own scaling examples in Section 2. This section builds that foundation properly from the ground up: what Docker and Kubernetes actually are made of, how their pieces talk to each other, and every core operation — build, run, push, deploy, scale, update, and roll back — shown both the **imperative** way (direct CLI commands) and the **declarative** way (YAML manifests applied to the cluster), since production teams use both and knowing when each fits matters as much as knowing the commands themselves.

11.1 Docker Architecture & Core Components

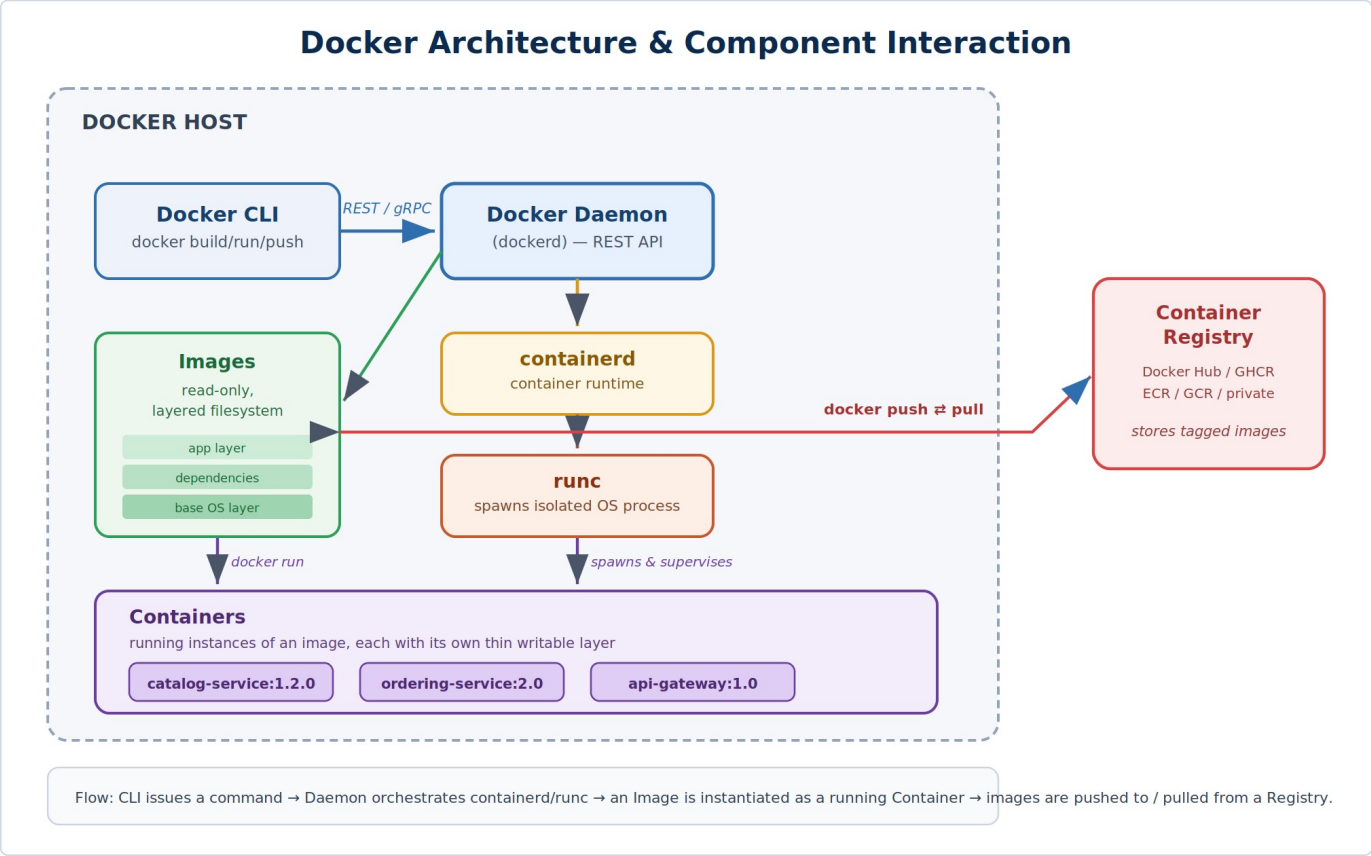


Fig. 9 — Docker CLI issues commands to the Docker Daemon over REST/gRPC; the Daemon orchestrates containerd and runc to actually run containers, manages layered Images, and pushes/pulls those images to and from a Container Registry.

Component	Role
Docker CLI	The `docker` command you type — a thin client that sends requests to the daemon over a REST API
Docker Daemon (dockerd)	The long-running background process that actually manages images, containers, networks, and volumes
containerd / runc	The lower-level runtime layer that actually creates and supervises the isolated OS process a container runs as
Image	An immutable, layered filesystem snapshot plus metadata (entrypoint, exposed ports, env defaults) — built once from a `Dockerfile`, run many times
Container	A running (or stopped) instance of an image, with its own thin writable layer on top of the image's read-only layers
Registry	Where built images are stored and fetched from — the handoff point between "I built this" and "any host can run this"

11.2 Hands-On: Build, Run, and Push

Dockerfile (catalog-service, multi-stage for a smaller final image)

```
FROM eclipse-temurin:17-jdk AS build
WORKDIR /app
COPY pom.xml .
COPY src ./src
RUN ./mvnw -q package -DskipTests

FROM eclipse-temurin:17-jre
WORKDIR /app
```

```
COPY --from=build /app/target/catalog-service.jar app.jar
EXPOSE 8081
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Imperative — direct CLI commands

```
# build: turns the Dockerfile + source into a tagged image
docker build -t registry.example.com/catalog-service:1.2.0 .

# run: starts a container from that image, mapping its port to the host
docker run -d --name catalog-service -p 8081:8081 \
  -e SPRING_DATA_MONGODB_URI=mongodb://mongo:27017/catalog_db \
  registry.example.com/catalog-service:1.2.0

# push: uploads the built image to the registry so other hosts/clusters can pull it
docker login registry.example.com
docker push registry.example.com/catalog-service:1.2.0
```

Declarative — the same result, expressed as a Compose file

docker-compose.yml

```
services:
  catalog-service:
    build: .
    image: registry.example.com/catalog-service:1.2.0
    ports: ["8081:8081"]
    environment:
      SPRING_DATA_MONGODB_URI: mongodb://mongo:27017/catalog_db
```

```
docker compose build          # equivalent to the imperative "docker build" above
docker compose up -d          # equivalent to the imperative "docker run" above
docker compose push           # equivalent to the imperative "docker push" above
```

> **The trade-off in one line:** imperative commands are faster to type for a one-off, ad hoc action; a declarative file is checked into version control, reviewed, and reproduced identically by anyone who runs it — which is why CI/CD pipelines (Section 11.7 onward) are built almost entirely on declarative definitions rather than a sequence of remembered CLI commands.

11.3 Kubernetes Architecture & Core Components

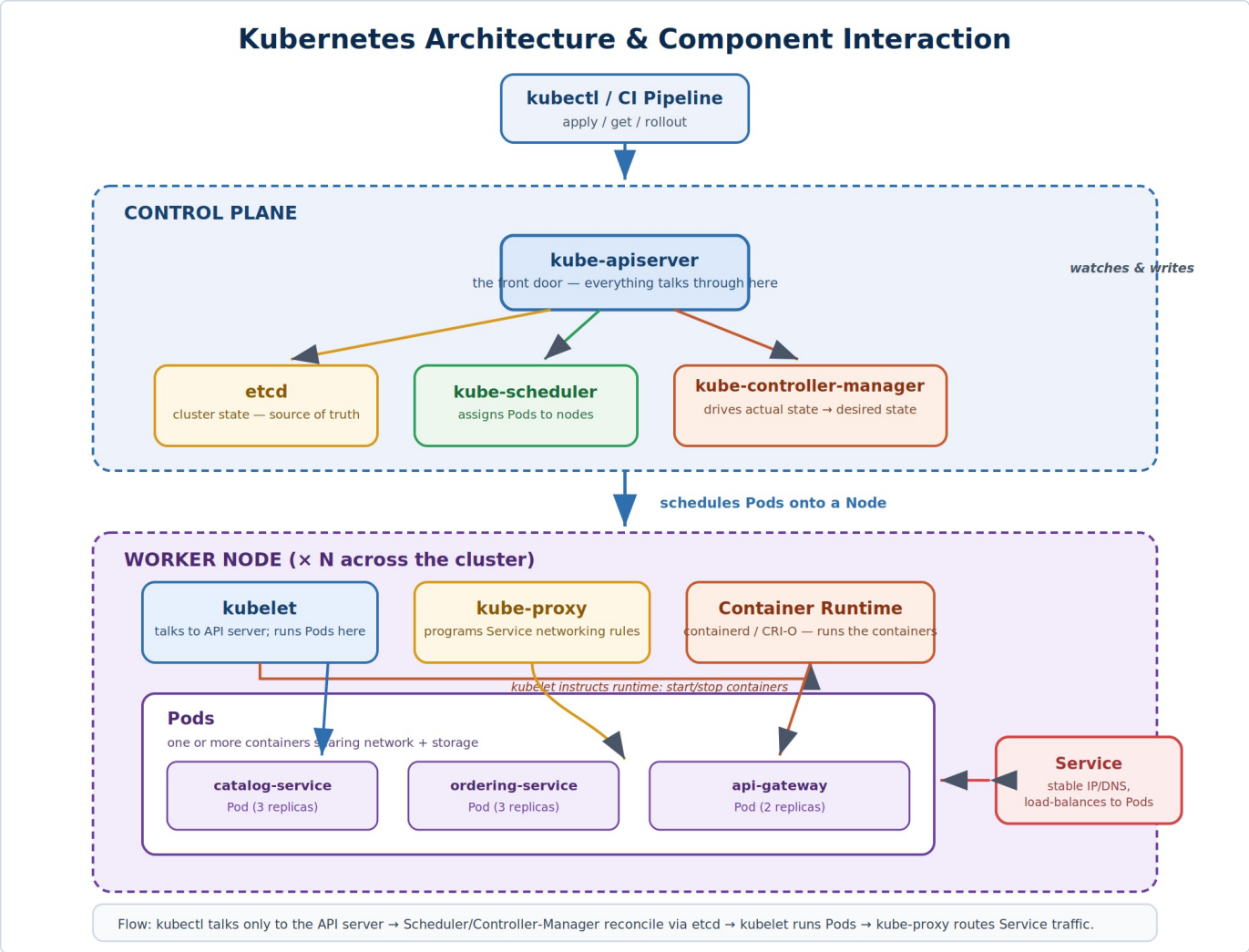


Fig. 10 — kubectl (or a CI pipeline) talks only to the API server; the Scheduler and Controller-Manager reconcile desired vs. actual state via etcd; the Scheduler assigns Pods onto a Worker Node, where kubelet runs them via the container runtime and kube-proxy programs the networking rules a Service uses to load-balance traffic to those Pods.

Object	What it's for
Pod	The smallest deployable unit — one or more tightly-coupled containers sharing a network namespace and storage volumes
Deployment	Declares "I want N replicas of this Pod spec running" and manages rolling updates/rollbacks to get there (Sections 11.5–11.6)
ReplicaSet	The mechanism a Deployment uses under the hood to actually maintain N running Pod replicas — rarely managed directly
Service	A stable network identity/IP + DNS name in front of a changing set of Pods, load-balancing traffic across whichever Pods currently match its selector
Ingress	HTTP(S) routing rules from outside the cluster to internal Services — the Kubernetes-native entry point in front of, or alongside, an API Gateway (Part I, Pattern 6)
ConfigMap / Secret	Externalized configuration and sensitive values (Part I, Pattern 9) injected into Pods as environment variables or mounted files
Namespace	A logical partition of one cluster — separating, for example, `staging` and `production` resources without needing separate clusters
HorizontalPodAutoscaler (HPA)	Watches a metric (CPU, custom metrics) and adjusts a Deployment's replica count automatically (Section 11.4)

How this maps onto everything built so far: each of this guide's Spring Boot services (catalog-service, ordering-service, api-gateway, and so on) becomes one Deployment + one Service in Kubernetes; the API Gateway pattern from Part I sits either as its own Deployment/Service or is replaced by/layered under an Ingress; ConfigMaps and Secrets are exactly Part I's Externalized Configuration pattern, now expressed as native Kubernetes objects instead of (or alongside) a Spring Cloud Config Server.

11.4 Hands-On: Deploying catalog-service — Imperative vs. Declarative

Imperative — one-off commands, useful for quick iteration or debugging

```
# create a Deployment directly from the CLI, no YAML file involved
kubectl create deployment catalog-service --image=registry.example.com/catalog-service:1.2.0 --replicas=3

# expose it as a Service
kubectl expose deployment catalog-service --port=8081 --target-port=8081 --type=ClusterIP

# quick imperative edits
```

```
kubectl scale deployment catalog-service --replicas=5
kubectl set env deployment/catalog-service SPRING_PROFILES_ACTIVE=production
```

Declarative — the production-standard approach, checked into Git

catalog-deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: catalog-service
  labels: { app: catalog-service }
spec:
  replicas: 3
  selector:
    matchLabels: { app: catalog-service }
  template:
    metadata:
      labels: { app: catalog-service }
    spec:
      containers:
        - name: catalog-service
          image: registry.example.com/catalog-service:1.2.0
          ports: [ { containerPort: 8081 } ]
          envFrom:
            - configMapRef: { name: catalog-service-config }
          readinessProbe:
            httpGet: { path: /actuator/health/readiness, port: 8081 }
            initialDelaySeconds: 10
          livenessProbe:
            httpGet: { path: /actuator/health/liveness, port: 8081 }
            initialDelaySeconds: 20
---
apiVersion: v1
kind: Service
metadata:
  name: catalog-service
spec:
  selector: { app: catalog-service }
  ports: [ { port: 8081, targetPort: 8081 } ]
  type: ClusterIP
```

```
kubectl apply -f catalog-deployment.yaml
kubectl get deployments,pods,svc -l app=catalog-service
```

> **Why declarative wins for anything beyond a quick experiment:** `kubectl apply` is idempotent — running it again with an unchanged file does nothing, and running it with a changed file reconciles the cluster to match, safely. Imperative commands like `kubectl create` fail if the resource already exists, and there's no file in Git recording exactly what was actually deployed. Every CI/CD pipeline in Sections 11.7 onward applies YAML manifests specifically because of this reproducibility.

11.5 Autoscaling — Imperative vs. Declarative

Section 2 introduced the HPA concept briefly; here's the full imperative/declarative pair for it, completing the pattern established throughout this section.

Imperative

```
kubectl autoscale deployment catalog-service --cpu-percent=65 --min=2 --max=15
kubectl get hpa catalog-service   # watch current replica count vs. target
```

Declarative

catalog-hpa.yaml

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: catalog-service-hpa
spec:
  scaleTargetRef: { apiVersion: apps/v1, kind: Deployment, name: catalog-service }
  minReplicas: 2
  maxReplicas: 15
  metrics:
    - type: Resource
      resource: { name: cpu, target: { type: Utilization, averageUtilization: 65 } }
```

```
kubectl apply -f catalog-hpa.yaml
```

```
# load-test to trigger a scale-up, then watch it happen
kubectl run load-generator --image=williamyeh/hey --restart=Never -- \
```

```
  hey -z 3m -c 100 http://catalog-service:8081/v1/products
watch kubectl get hpa catalog-service
# CURRENT column climbs from 2 toward 15 as CPU utilization crosses the 65% threshold
```

11.6 Rolling Updates — Imperative vs. Declarative

A rolling update replaces old Pods with new ones gradually, keeping the service available throughout — no big-bang cutover, no downtime window.

Declaring the rollout strategy up front (part of the Deployment spec)

```
spec:
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1          # at most 1 extra Pod above the desired replica count during the rollout
      maxUnavailable: 0     # never drop below the desired replica count -- zero-downtime
```

Imperative — trigger the rollout directly

```
kubectl set image deployment/catalog-service catalog-service=registry.example.com/catalog-service:1.3.0
kubectl rollout status deployment/catalog-service
# Kubernetes replaces Pods one (or maxSurge) at a time, waiting for each new Pod's readinessProbe
# to pass before moving on to the next -- exactly the readinessProbe defined in Section 11.4
```

Declarative — change the manifest, re-apply

```
# edit catalog-deployment.yaml: image: registry.example.com/catalog-service:1.3.0
kubectl apply -f catalog-deployment.yaml
kubectl rollout status deployment/catalog-service
```

11.7 Rollback

```
# see the rollout history Kubernetes has been tracking all along
kubectl rollout history deployment/catalog-service

# roll back to the immediately previous revision
kubectl rollout undo deployment/catalog-service

# or roll back to a SPECIFIC earlier revision
kubectl rollout undo deployment/catalog-service --to-revision=3
```

> **Why this works without you doing anything extra:** every `kubectl apply` or `kubectl set image` against a Deployment automatically creates a new ReplicaSet and keeps the old one (scaled to zero) around as revision history. Rollback is simply scaling the old ReplicaSet back up and the new one down — the exact same rolling-update machinery from Section 11.6, just running in reverse.

12. CI/CD Pipeline: Canary & Blue-Green Deployment Strategies

12.1 CI/CD Pipeline Overview

Everything in Sections 11.1–11.7 is a set of manual steps. A CI/CD pipeline automates that exact sequence — build, test, push, deploy — triggered by a Git commit, with no human running `kubectl apply` by hand in production.

Git push ⟶ CI: build + test + docker build + docker push │ ▼ CD: kubectl apply (or Helm/Argo CD sync) │ ┌──────────────┐ ▼ ▼ Canary rollout Blue-Green cutover (Section 11.9) (Section 11.10)

.github/workflows/deploy.yml — a minimal CI/CD pipeline (GitHub Actions)

```
name: build-and-deploy
on: { push: { branches: [main] } }
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Build & test
        run: ./mvnw -q clean verify
      - name: Build & push image
        run: |
          IMAGE=registry.example.com/catalog-service:${{ github.sha }}
          docker build -t $IMAGE .
          docker push $IMAGE
      - name: Deploy (declarative apply, image tag templated per commit)
        run: |
          sed -i "s#catalog-service:.*#catalog-service:${{ github.sha }}#" k8s/catalog-deployment.yaml
          kubectl apply -f k8s/catalog-deployment.yaml
          kubectl rollout status deployment/catalog-service
```

12.2 Canary Deployment — Hands-On

A canary release sends a small percentage of live traffic to the new version while the rest keeps hitting the stable version — watching real production metrics on that small slice before committing to a full rollout. Plain Kubernetes can approximate this by running two Deployments behind one Service and controlling the traffic split via relative replica counts; a service mesh or a dedicated tool (Argo Rollouts, covered below) makes the split explicit and precise rather than approximate.

Approach A — Native Kubernetes (approximate split via replica ratio)

Two Deployments, one Service selecting both via a shared label

```
apiVersion: apps/v1
kind: Deployment
metadata: { name: catalog-service-stable }
spec:
  replicas: 9                          # 90% of traffic, approximately, since the Service load-balances evenly
  selector: { matchLabels: { app: catalog-service, track: stable } }
  template:
    metadata: { labels: { app: catalog-service, track: stable } }
    spec:
      containers: [ { name: catalog-service, image: registry.example.com/catalog-service:1.2.0 } ]
---
apiVersion: apps/v1
kind: Deployment
metadata: { name: catalog-service-canary }
spec:
  replicas: 1                          # 10% of traffic, approximately
  selector: { matchLabels: { app: catalog-service, track: canary } }
  template:
    metadata: { labels: { app: catalog-service, track: canary } }
    spec:
      containers: [ { name: catalog-service, image: registry.example.com/catalog-service:1.3.0 } ]
---
apiVersion: v1
kind: Service
metadata: { name: catalog-service }
spec:
  selector: { app: catalog-service }   # matches BOTH tracks -- traffic split follows the replica ratio above
  ports: [ { port: 8081, targetPort: 8081 } ]
```

```
# promote the canary once its metrics look healthy: scale it up, scale stable down, then retire stable
kubectl scale deployment catalog-service-canary --replicas=10
kubectl scale deployment catalog-service-stable --replicas=0
kubectl delete deployment catalog-service-stable
```

Approach B — Argo Rollouts (precise, percentage-based, automated canary)

rollout.yaml — replaces the plain Deployment with a Rollout resource

```
apiVersion: argoproj.io/v1alpha1
kind: Rollout
metadata: { name: catalog-service }
spec:
  replicas: 10
  strategy:
    canary:
      steps:
        - setWeight: 10        # step 1: exactly 10% of traffic to the new version
        - pause: { duration: 5m }
        - setWeight: 50        # step 2: 50%, after step 1 held steady with no manual intervention
        - pause: { duration: 5m }
        - setWeight: 100       # step 3: full rollout
  selector: { matchLabels: { app: catalog-service } }
  template:
    metadata: { labels: { app: catalog-service } }
    spec:
      containers: [ { name: catalog-service, image: registry.example.com/catalog-service:1.3.0 } ]
```

```
kubectl apply -f rollout.yaml
kubectl argo rollouts get rollout catalog-service --watch
# shows live traffic-weight percentage moving through 10% -> 50% -> 100%, pausing at each step

# abort and roll back automatically if metrics look wrong mid-rollout
kubectl argo rollouts abort catalog-service
```

12.3 Blue-Green Deployment — Hands-On

Blue-Green keeps two complete, independent environments — "Blue" (currently live) and "Green" (the new version) — and cuts traffic over from one to the other *entirely*, in one atomic switch, rather than gradually. The immediate rollback is simply switching back, since the old environment is left running, untouched, until the new one is confirmed healthy.

Two Deployments (blue and green), Service selector is the actual switch

```
apiVersion: apps/v1
kind: Deployment
metadata: { name: catalog-service-blue }
spec:
  replicas: 10
  selector: { matchLabels: { app: catalog-service, version: blue } }
  template:
    metadata: { labels: { app: catalog-service, version: blue } }
    spec:
      containers: [ { name: catalog-service, image: registry.example.com/catalog-service:1.2.0 } ]
---
apiVersion: apps/v1
kind: Deployment
metadata: { name: catalog-service-green }
spec:
  replicas: 10
  selector: { matchLabels: { app: catalog-service, version: green } }
  template:
    metadata: { labels: { app: catalog-service, version: green } }
    spec:
      containers: [ { name: catalog-service, image: registry.example.com/catalog-service:1.3.0 } ]
---
apiVersion: v1
kind: Service
metadata: { name: catalog-service }
spec:
  selector: { app: catalog-service, version: blue }   # <-- THIS line is the entire cutover
  ports: [ { port: 8081, targetPort: 8081 } ]
```

Imperative — the actual cutover, and the rollback

```
# cut ALL live traffic over to green in one atomic edit
kubectl patch service catalog-service -p '{"spec":{"selector":{"app":"catalog-service","version":"green"}}}'

# smoke-test green directly, bypassing the Service, before or immediately after cutover
kubectl port-forward deployment/catalog-service-green 8081:8081

# rollback: an equally instant, equally atomic switch back to blue
kubectl patch service catalog-service -p '{"spec":{"selector":{"app":"catalog-service","version":"blue"}}}'

# only once green has run confidently in production for a while, retire blue
kubectl delete deployment catalog-service-blue
```

> **Canary vs. Blue-Green, when to reach for which:** Canary trades a slower rollout for early, real-traffic signal on a small population before committing further — better when you want to catch subtle issues (error rate creep, latency regression) before they affect everyone. Blue-Green trades that gradual-signal benefit for the simplest possible rollback story — one atomic switch back — better when a release is either clearly fine or clearly broken and you want zero ambiguity about which environment is live at any moment.

12.4 Putting the Whole Pipeline Together

git push (main) │ ▼ CI: mvn verify → docker build → docker push (Section 11.8) │ ▼ CD: kubectl apply -f rollout.yaml (Argo Rollouts Canary, Section 11.9) │ ┌─ 10% for 5min → 50% for 5min → 100% ─┐ │ │ (auto-analyzed against error-rate / latency metrics) │ ▼ └──────────────────────────────────────┘ Fully promoted, OR auto-rolled-back if metrics regress mid-canary │ ▼ (For a major/riskier release instead) kubectl patch service ... version: green (Blue-Green atomic cutover, Section 11.10, with blue kept warm for instant rollback)

> **Everything in this section, tied back to the rest of the guide series:** the Deployments and Services here host every Spring Boot service built across Parts I–IV; the readinessProbe/livenessProbe endpoints are Spring Boot Actuator, already wired up wherever Circuit Breaker (Part I/IV) or Resilience patterns were built; the HPA in Section 11.5 is the exact mechanism Part II Section 10's capacity-planning table assumed; and the CI/CD pipeline here is what actually ships every pattern in this guide series to production, safely, automatically, and reversibly.

13. What's Next

Where This Foundation Leads

The architecture built in Section 6 — clients calling Catalog, Shopping Cart, Discount, and Ordering directly, each with its own database — is a correct, working starting point. But it also has an immediately visible gap: the client applications talk directly to every individual microservice, which means every client needs to know the network location of every service, and any change to that internal topology becomes a client-side change. That's precisely the seam this guide's series picks up from here.

Next Step	What It Solves	Where to Find It
API Gateway	One stable entry point instead of direct client-to-service coupling; a centralized place for auth, rate limiting, and routing	Part I, Pattern 6 (with the Facade pattern analogy, feature breakdown, and Ocelot comparison)
Backend for Frontend	Avoiding one overloaded, business-logic-laden gateway as client types and needs diverge	Part I, Pattern 5
Sync vs. Async communication, REST vs. gRPC	Choosing the right protocol per boundary rather than defaulting to one style everywhere	Part III, in full
CQRS, Event Sourcing, Sagas	Handling data consistency and cross-service transactions once a single request touches more than one service's database	Part I, Patterns 2–4; Part II, in full
Circuit Breaker, Retry, Rate Limiter, Bulkhead	Containing network failures, transient errors, overload, and resource exhaustion as the service count grows	Part I, Pattern 8; Part IV, Sections 8–9 (this document)
Caffeine & Redis Caching, Eviction Policies (LRU/LFU/TTL)	Avoiding repeated calls to a dependency entirely, reducing both latency and the load a rate limiter would otherwise need to reject	Part IV, Section 10 (this document)
Docker, Kubernetes, Autoscaling, Rolling Updates/Rollback, CI/CD, Canary & Blue-Green	Actually shipping every pattern in this guide series to production, safely and automatically	Part IV, Sections 11–12 (this document)
Service Registry, Service Aggregator	Removing chatty, chained synchronous calls and fixed service addresses	Part I, Bonus A
The Outbox Pattern	Guaranteeing a database write and an event publish happen atomically	Part I, Bonus D
Event Hub, Stream Processing, Container Orchestration	Scaling the whole platform toward millions of concurrent requests	Part II, Sections 6–10

The path this guide series takes, start to finish: Part IV (this document) establishes why microservices exist, what they're built from, what a correct starting architecture looks like, and how to make every inter-service call resilient — with hands-on code at every one of its own sections, not just theory. Part I takes that starting architecture and evolves it pattern by pattern — API Gateway, BFF, CQRS, Event Sourcing, Sagas, and more — each introduced exactly where the previous stage's limits start to show. Part II goes deep on the event-driven end state that Part I's patterns converge toward. Part III fills in the communication-protocol layer underneath all of it. Read in that order, each part answers the question the previous one leaves open.